

컴퓨터 비전

Computer Vision

- Feature Extraction and Matching -

동아대학교 소프트웨어대학 AI학과
2026년 1학기

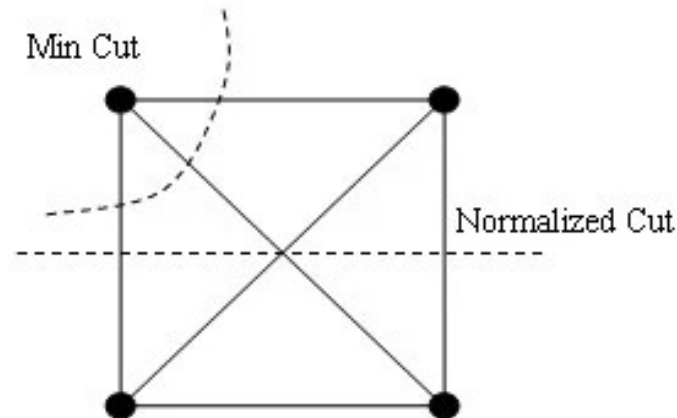
임한신

최적화 분할

- 영상의 그래프 표현
 - Normalized cut(N-cut, 정규화 절단)알고리즘
 - cut은 영상을 두 영역으로 분할했을 때 분할의 좋은 정도를 측정해 주는 목적 함수
 - C_1 과 C_2 가 클수록 둘 사이에 간선이 많아 cut은 덩달아 커지므로 cut을 사용한 분할 알고리즘은 영역을 자잘하게 분할하는 경향이 있음

$$cut(C_1, C_2) = \sum_{v_p \in C_1, v_q \in C_2} s_{pq}$$

- N-cut은 cut을 정규화하여 영역의 크기에 중립이 되게 해줌
- Spectral clustering 기법을 영상에 적용



대응점 문제(correspondence problem)

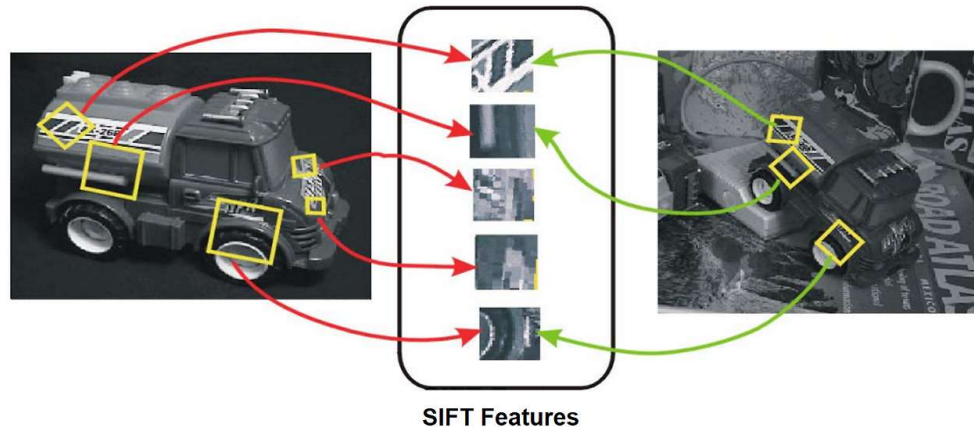
- 이웃한 영상에 나타난 같은 물체의 같은 곳을 쌍으로 맺는 일
- 파노라마, 물체 인식, 물체 추적, 스테레오 비전, 카메라 캘리브레이션 등에 필수
 - 예) 파노라마 영상 제작에서 봉합되는 지점을 결정하는데 필수



파노라마 : 여러 시점을 하나의 넓은 시야로 합친 영상

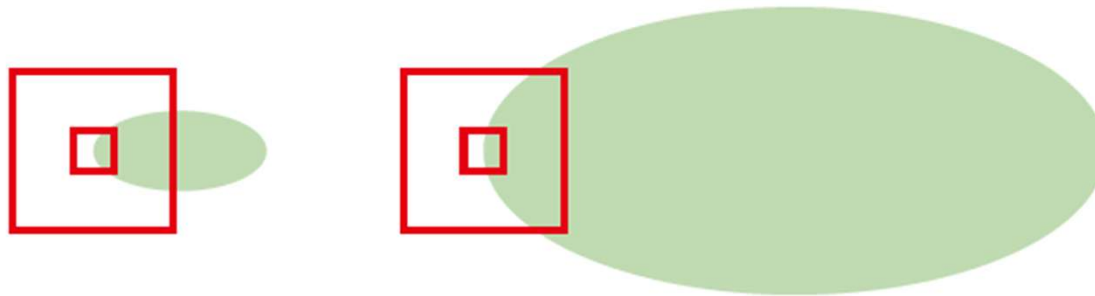
특징 검출

- 좋은 특징(feature)의 조건
 - 불변성(Invariance)
 - 크기, 회전, 각도, 조명등의 변화에도 특징의 값이 크게 변하지 않아야 함
 - 변별성(Distinctiveness)
 - 코너나 패턴이 많은 영역처럼 다른 영역과 명확히 달라야 함
 - 재현력(Repeatability)
 - 다른 조건(다른 영상)에서도 특징으로 검출될 수 있어야 함
 - 지역성(Locality)
 - 특징점이 영상의 좁고 국소적인 영역의 정보만을 바탕으로 정의되어야 함



특징 검출

- 해리스 코너 검출(Harris corner detection) 알고리즘의 분석
 - 물체의 실제 코너 뿐 아니라 블롭(blob)에서도 검출
 - 블롭(blob) : 주변과 색상이나 밝기가 뚜렷하게 대비되는, 비슷한 성질을 가진 픽셀들의 뭉치
 - 이동과 회전에 불변
 - 스케일에는 불변 아님



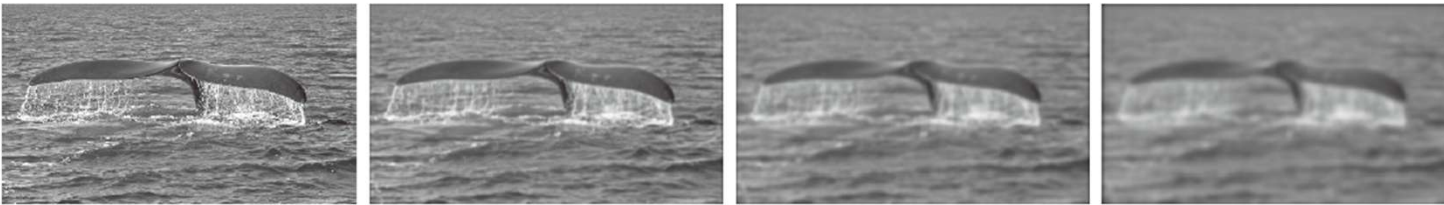
객체의 크기에 따라 적절한 마스크의 크기가 다름

특징 검출

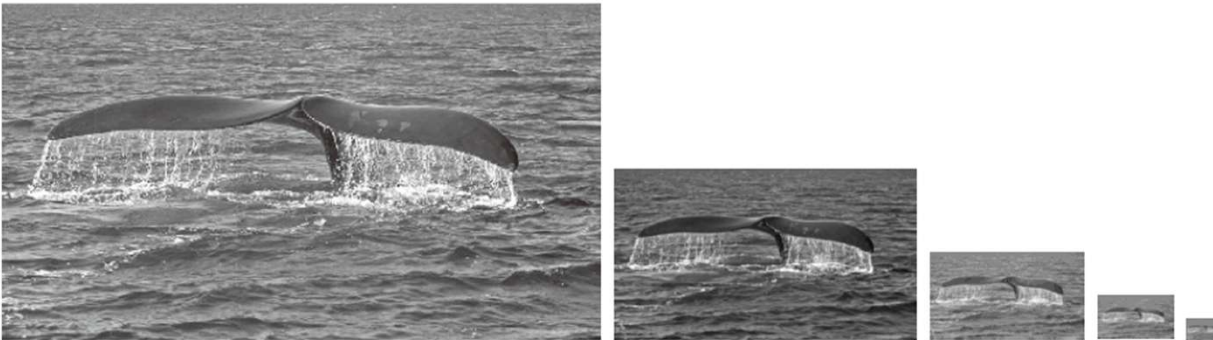
- 스케일 불변한 지역 특징
 - 사람은 스케일 불변인 특징 사용
 - 거리에 상관없이 같은 물체를 같다고 인식
 - 거리에 따라 세세한 내용에만 차이가 있음
- 스케일 공간(scale space)이론
 - 영상을 단일 해상도가 아닌, 다양한 크기(스케일)의 집합으로 다루어 사물을 인식하려는 방법
 - 영상을 단계적으로 흐리게(블러 처리) 하거나 크기를 줄여가며, 모든 스케일에서 공통적으로 발견되는 특징을 찾아냄

특징 검출

- 다중 스케일 영상 구현 방법
 - 입력은 영상 한 장. 여러 거리에서 보았을 때 나타나는 현상을 모사
 - 가우시안 스무딩과 피라미드 방법



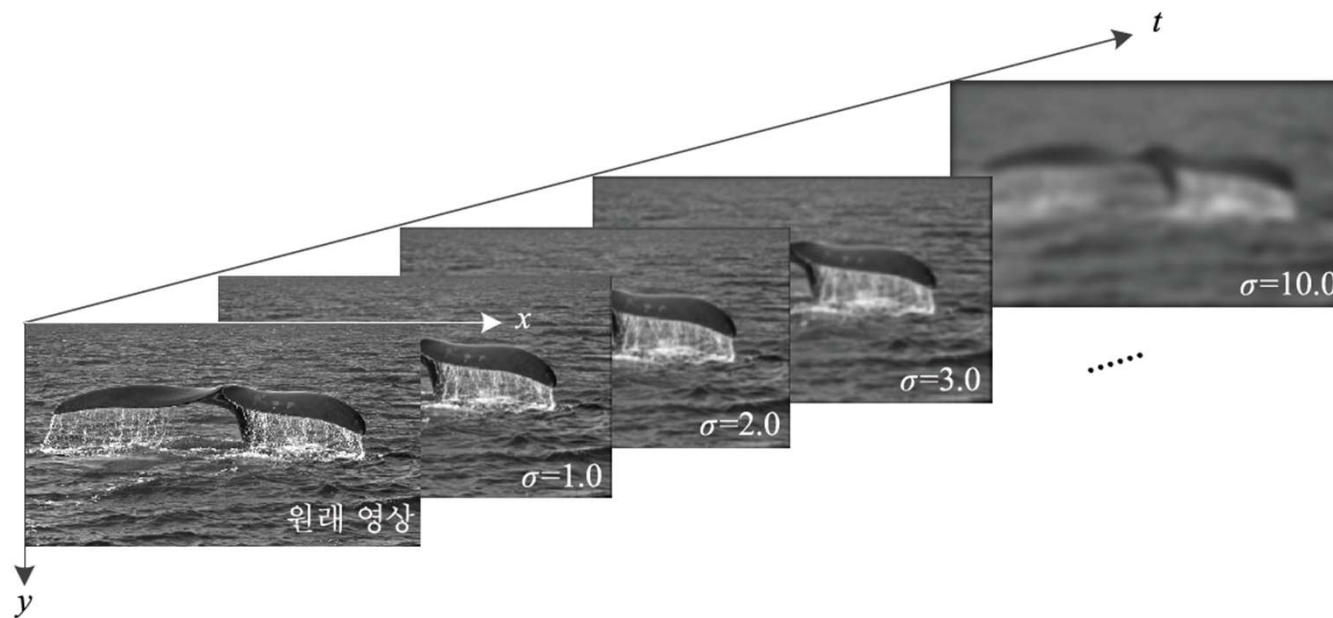
가우시안 스무딩(σ 값을 변화시키면서 스무딩)



피라미드 방법(영상의 크기를 줄임(downscaling))

특징 검출

- 다중 스케일 영상 구현 방법
 - 가우시안 스무딩은 스케일을 연속값으로 조절할 수 있는 장점이 있음



SIFT(Scale-Invariant Feature Transform)

- 이미지의 크기(Scale)가 변하거나 회전(Rotation)해도 변하지 않는 특징을 추출하고 기술(Description)하는 알고리즘
- 스케일 공간(scale space)이론에 기반하여 특징을 추출
- 일반적인 코너 검출기(Harris Corner 등)는 영상을 조금만 키우거나 돌려도 코너를 놓치는 경우가 많지만 SIFT는 다음 상황에서도 안정적인 특징을 찾음
 - 크기 변화 (Scale): 멀리서 찍든 가까이서 찍든 동일한 특징을 찾음
 - 회전 (Rotation): 카메라를 옆으로 돌려 찍어도 동일한 특징을 찾음.
 - 조명 변화 (Illumination): 사진이 조금 어둡거나 밝아져도 특징을 유지함
- 이전에는 특허권이 있어 상업적 이용이 제한적이었으나 2020년에 특허가 만료되어 자유롭게 사용 가능

SIFT(Scale-Invariant Feature Transform)

- SIFT는 크게 특징 검출(Detection) 단계와 특징 기술(Description) 단계로 나뉨
- 특징 검출 (Feature Detection)
 - 영상 내에서 크기 변화나 노이즈에 강인한 키폰트(Keypoints)를 찾는 과정
- 특징 기술 (Feature Description)
 - 찾아낸 특징을 지문처럼 고유한 숫자로 표현하여, 나중에 다른 영상에서도 매칭할 수 있게 만드는 과정

SIFT(Scale-Invariant Feature Transform)

- 특징 검출(Detection) 단계

1. 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 방법을 결합해 사용

2. 다중 스케일 영상에 미분 적용

- 정규 라플라시안은 시간이 많이 걸려 유사한 DoG(Difference of Gaussian) 사용
- DoG는 이전에 만든 가우시안 영상에서 이웃한 가우시안을 빼면 되기 때문에 획기적으로 빠름

3. 극점 검출

- DoG 영상들 사이에서 주변 26개 픽셀(같은 층 8개 + 위층 9개 + 아래층 9개)보다 값이 크거나 작은 지점을 특징 후보로 선택함으로써 스케일 불변성을 확보

4. 키포인트 정밀화

- 믿을만한 특징만 남기고, 그 위치를 소수점 단위(Sub-pixel)로 정확하게 조정하는 과정

SIFT(Scale-Invariant

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 생성

- 다중 스케일 영상에 DoG 적용

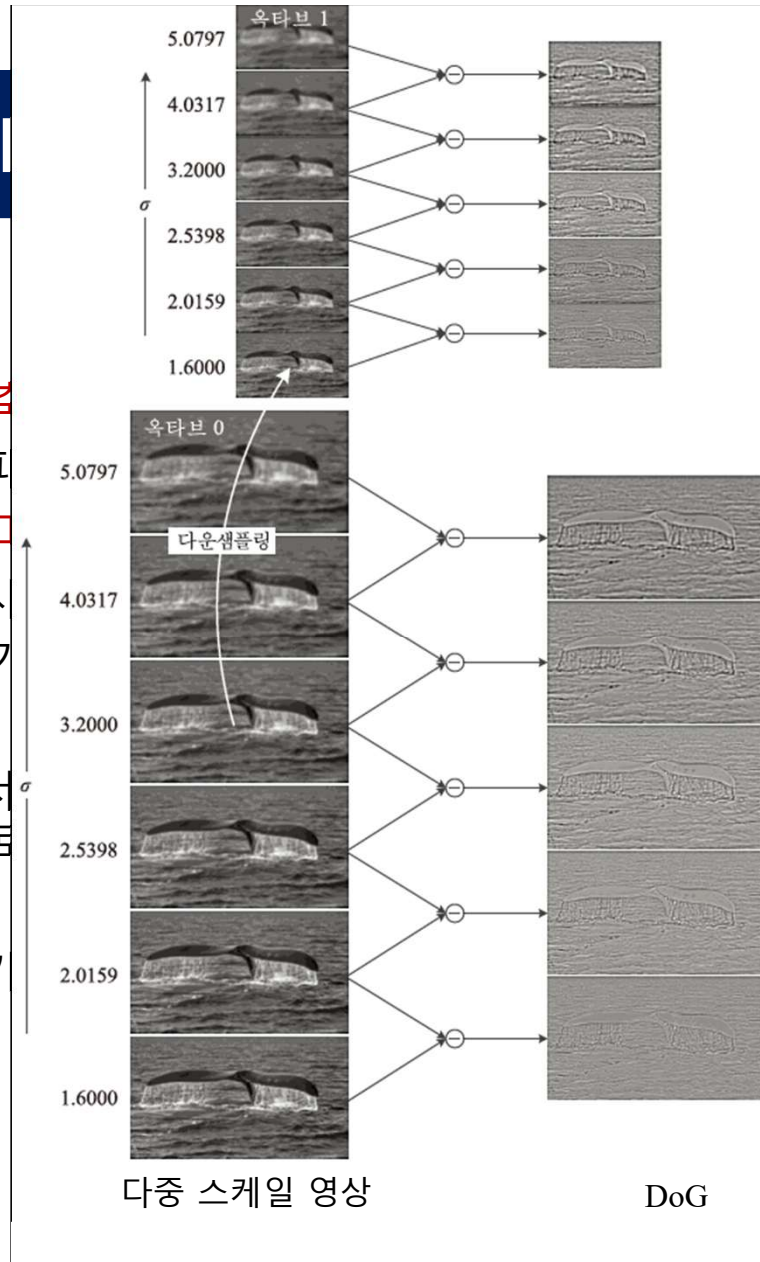
- 정규 라플라시안은 사용
- DoG는 이전에 만든 가우시안 피라미드에서 차분

- 극점 검출

- DoG 영상들 사이에서 극점을 찾아
- 은 지점을 특징 후보로

- 키포인트 정밀화

- 믿을만한 특징만 남기



(o : octave, i : interval)

Gaussian) 사용

! 되기 때문에 획기적으로 빠름

아래층 9개)보다 값이 크거나 작

확하게 조정하는 과정

SIFT(Scale-Invariant

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 생성

- 다중 스케일 영상에 DoG 적용

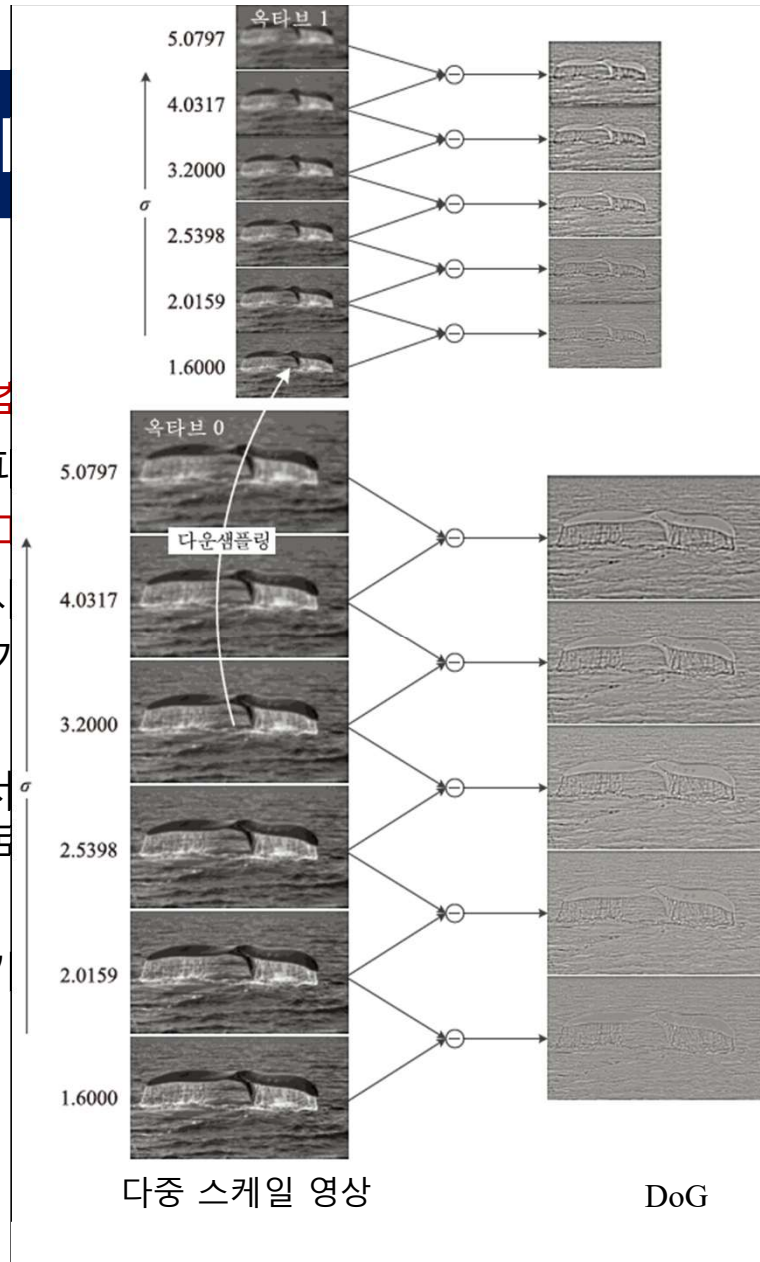
- 정규 라플라시안은 사용
- DoG는 이전에 만든 가우시안 피라미드에서 생성

- 극점 검출

- DoG 영상들 사이에서 극점을 찾아 특징 후보로 선정

- 키포인트 정밀화

- 믿을만한 특징만 남기



(o : octave, i : interval)

$o=0$ 이 원본 크기라면

$o=1$ 은 가로세로가 1/2로 줄어든 영상

$o=2$ 는 1/4로 줄어든 영상

i : 각 octave(o) 당 DoG의 층

(Gaussian) 사용

! 되기 때문에 획기적으로 빠름

(아래층 9개)보다 값이 크거나 작

확하게 조정하는 과정

SIFT(Scale-Invariant Feature Transform)

- 특징 검출(Detection) 단계

- 다중 스케일 영상 구축

- 가우시안 스무딩과 피라미드 방

- 다중 스케일 영상에 미분 적용

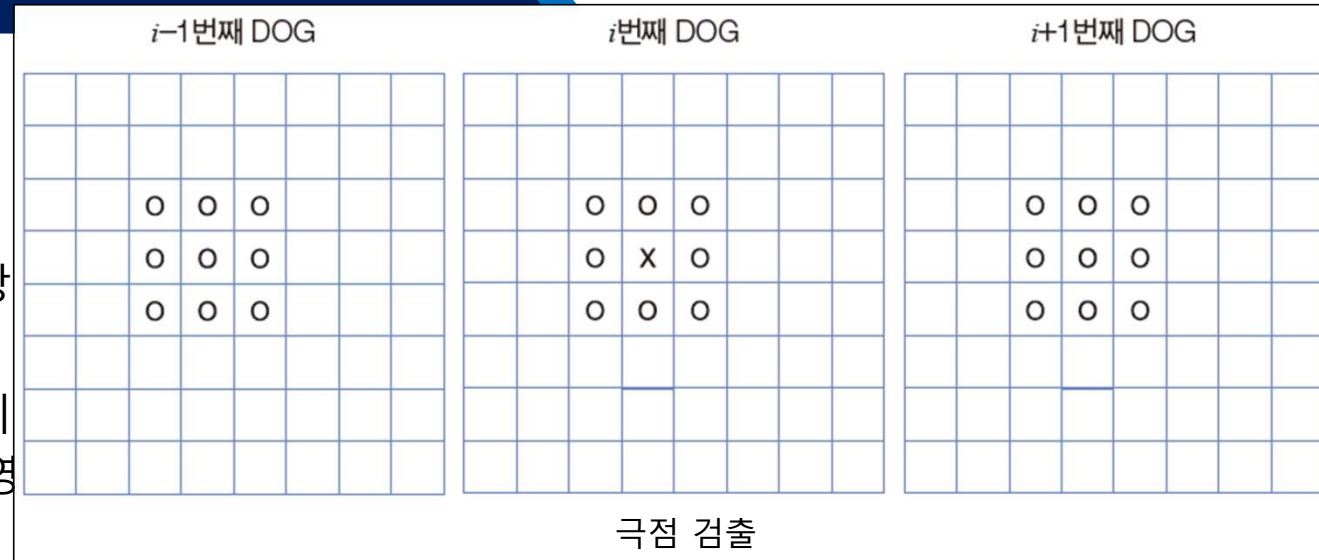
- 정규 라플라시안은 시간이 많이
- DoG는 이전에 만든 가우시안 영

- 극점 검출

- DoG 영상들 사이에서 주변 26개 픽셀(같은 층 8개 + 위층 9개 + 아래층 9개)보다 값이 크거나 작은 지점을 특징 후보로 선택함으로써 스케일 불변성을 확보

- 키포인트 정밀화

- 믿을만한 특징만 남기고, 그 위치를 소수점 단위(Sub-pixel)로 정확하게 조정하는 과정



DoG 실습

```
import urllib.request

from google.colab.patches import cv2_imshow

# 영상 로드 및 흑백 변환

url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'
urllib.request.urlretrieve(url, 'butterfly.jpg')
img = cv2.imread('butterfly.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 스케일 공간 설정 파라미터

s = 3
num_octaves = 4 # 총 4개의 옥타브 생성
sigma0 = 1.6
k = 2**(1.0/s) # 스케일 배수

# 스케일 공간 생성 (Gaussian Pyramid & DoG Pyramid)
gaussian_pyr = []
dog_pyr = []
```

```
# 원본 이미지의 초기 가우시안 필터링
base_img = cv2.GaussianBlur(gray, (0, 0), sigmaX=sigma0, sigmaY=sigma0)
gaussian_pyr.append([base_img])

for o in range(num_octaves):
    current_octave_gaussians = [base_img]
    current_octave_dogs = []

    # 하나의 옥타브 내에서 s+3개의 가우시안 이미지를 만들
    for i in range(1, s+3):
        # 이전 층의 sigma에 k를 곱해 가며 흐리게 만들.
        prev_sigma = sigma0 * (k ** (i-1))
        gaussian_img = cv2.GaussianBlur(current_octave_gaussians[-1], (0, 0),
sigmaX=prev_sigma)
        current_octave_gaussians.append(gaussian_img)

    # 인접한 가우시안 이미지의 차이 (DoG) 계산(high_sigma - low_sigma 순서)
    dog_img = current_octave_gaussians[i] - current_octave_gaussians[i-1]
    current_octave_dogs.append(dog_img)
```

DoG 실습

```
gaussian_pyr.append(current_octave_gaussians)
dog_pyr.append(current_octave_dogs)

# 다음 옥타브를 위해 크기를 절반으로 줄임
base_img = cv2.resize(current_octave_gaussians[s], (0, 0), fx=0.5, fy=0.5,
interpolation=cv2.INTER_NEAREST)

# 결과 시각화
print(f'--- SIFT DoG Pyramid (Octaves: {num_octaves}, Intervals: {s}) ---')
for o in range(num_octaves):
    num_dogs = len(dog_pyr[o])
    viz_list = []
    for d_img in dog_pyr[o]:
        # 차이값이 매우 작으므로 잘 보이게 정규화
        norm_img = cv2.normalize(d_img, None, 0, 255, cv2.NORM_MINMAX)
        viz_list.append(norm_img)

octave_viz = np.hstack(viz_list)
print(f'\n[Octave {o}] (Size: {octave_viz.shape[1]}x{octave_viz.shape[0]})')
cv2_imshow(octave_viz)
```

과제 #1

1. 현재까지 실습 코드(DoG까지)를 모두 수행하고 결과 출력 (1점)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리
2. 영상을 영상의 중심(height/2, width/2)을 축으로 임의의 각도(θ)만큼 회전 및 (a, b)만큼 이동 하는 프로그램(1.5점, 오실행 1점, 코드만 제출 0.5점)
 - Butterfly 영상을 입력으로 실행
 - Butterfly 영상을 225도 만큼 회전하고 (80, 80)만큼 이동한 결과가 출력되도록 할 것
 - 원래의 영상 크기보다 변환공간을 충분히 넓게 잡아야 회전했을 때 영상이 잘리지 않음
 - 영상의 기하 변환은 픽셀단위로 수행되어야 함(Backward mapping 방식으로 수행, OpenGL 변환함수 변환X)
 - 각 수행마다 코드에는 어떤 과정인지 주석 처리

제출 방법 : LMS로 압축 파일 제출

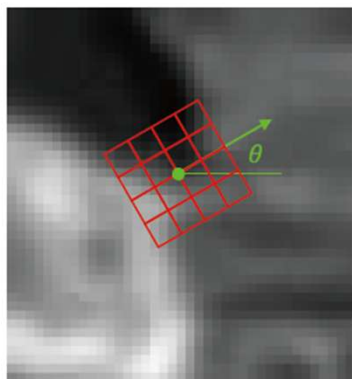
- 코랩에서 다음과 같은 이름을 가진 노트북 파일을 생성한다.
ComputerVision_1stHW_Class본인소속반_학번_영문이름.ipynb
예) ComputerVision_1stHW_Class1_24xxxxx_HanshinLim.ipynb
- 생성된 코랩 노트북 파일(.ipynb)에서 코드를 구현 및 결과를 출력한다. 결과 출력시 어떤 결과인지를 명시한다.
- Due : 4월 8일 밤 12시 (이후 제출 0점)

SIFT(Scale-Invariant Feature Transform)

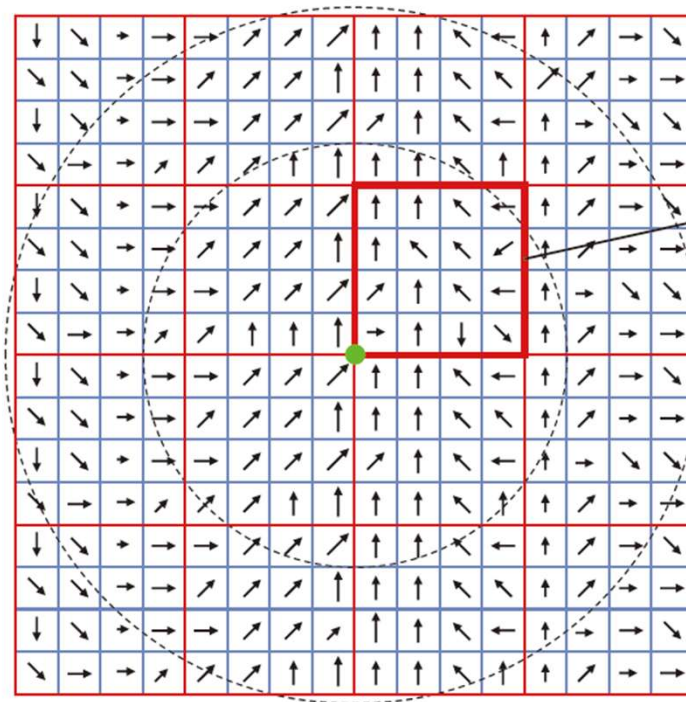
- 특징 기술(Description) 단계
 - 특징점 주위를 살펴 풍부한 정보를 가진 기술자 추출
 - 불변성 달성이 매우 중요
 - 기술자 추출 알고리즘
 1. o 와 i 로 가장 가까운 가우시안을 결정하고 거기서 기술자 추출(스케일 불변성 달성)
 2. 기준 방향을 정하고 기준 방향을 중심으로 특징을 추출(회전 불변성 달성)
 3. 기술자 x 를 단위 벡터로 바꿈(조명 불변성 달성)

SIFT(Scale-Invariant Feature Transform)

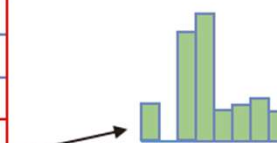
- SIFT 기술자 추출



지배적인 방향의 윈도우



16x16 부분 영역 샘플링과 기술자 추출



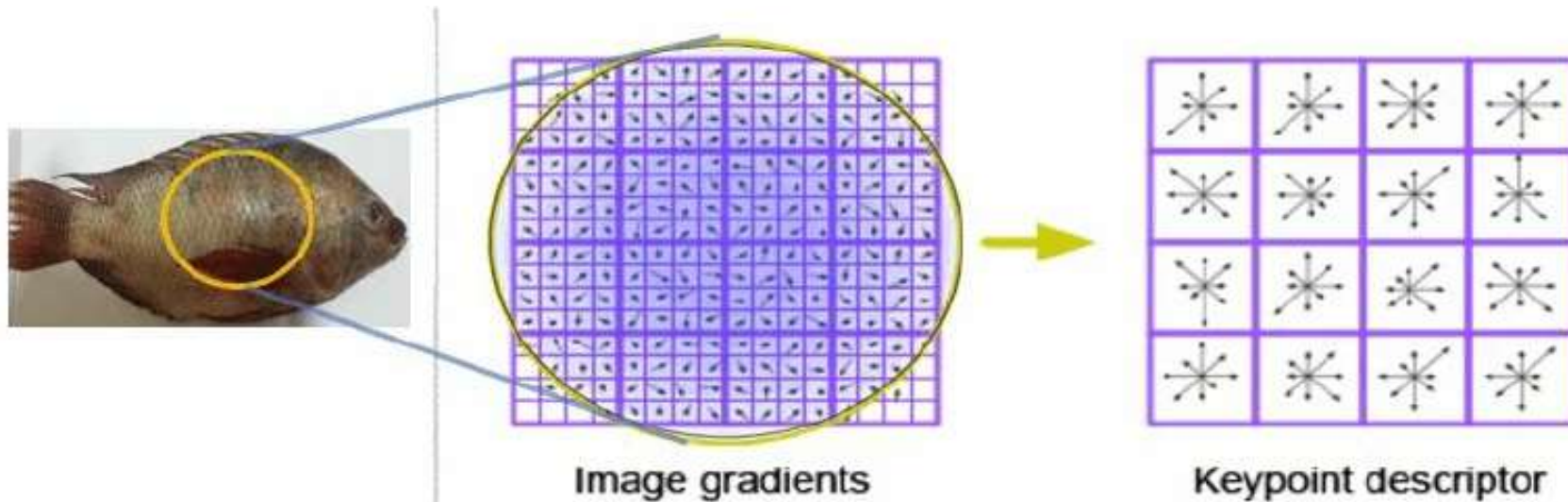
최종 $16 \times 8 = 128$ 차원의 기술자를 얻음

● 특징점의 중심 (y, x)

○ 가우시안 가중치

SIFT(Scale-Invariant Feature Transform)

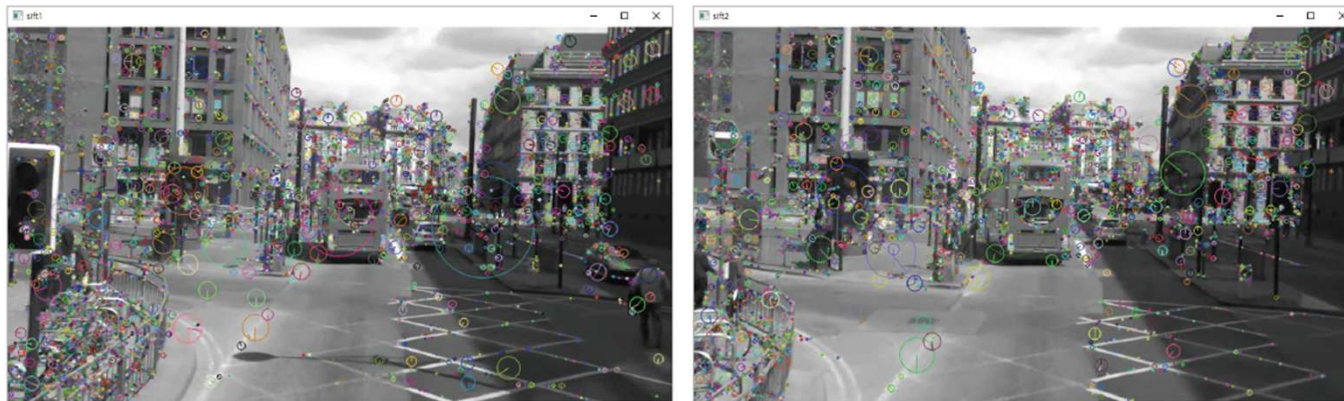
- SIFT 기술자 추출



SIFT 기술자 추출 예시

매칭(Matching)

- 매칭 문제의 이해
 - 두 영상에서 추출한 기술자 집합 $A=\{a_1, a_2, \dots, a_m\}$ 와 $B=\{b_1, b_2, \dots, b_n\}$ 에서 같은 물체의 같은 곳에서 추출된 쌍을 모두 찾는 문제
 - 매칭을 적용하는 다양한 상황
 - 물체 인식에서는 물체의 모델 영상이 A 이고 장면 영상이 B
 - 물체 추적이나 스테레오는 두 영상이 동등한 입장
- 매칭 문제의 어려움
 - 두 영상 모두 특징이 많아 후보 쌍이 아주 많은
 - 기술자에 잡음이 섞여있음



매칭(Matching)

- 두 기술자(descriptor) $\mathbf{a}=[a_1, \dots, a_d]$, $\mathbf{b}=[b_1, \dots, b_d]$ 간 거리 계산 (SIFT의 경우 $d=128$)
 - 유클리디안(Euclidean, L2) 거리

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{k=1, d} (a_k - b_k)^2} = \|\mathbf{a} - \mathbf{b}\| \quad (5.12)$$

- 매칭 전략

- 최근접 이웃: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 를 찾고 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 가 아래 식을 만족시키면 매칭쌍으로 간주

$$d(\mathbf{a}_i, \mathbf{b}_j) < T \quad T: \text{임계값}$$

- 최근접 이웃 거리 비율: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 와 두 번째로 가까운 $\mathbf{b}_k$ 를 찾음. $\mathbf{b}_j$ 와 $\mathbf{b}_k$ 가 아래 식을 만족시키면 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 는 매칭쌍으로 간주

$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$

매칭(Matching)

- 두 기술자(descriptor) $\mathbf{a}=[a_1, \dots, a_d]$, $\mathbf{b}=[b_1, \dots, b_d]$ 간 거리 계산 (SIFT의 경우 $d=128$)
 - 유클리디안(Euclidean, L2) 거리

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{k=1, d} (a_k - b_k)^2} = \|\mathbf{a} - \mathbf{b}\|$$

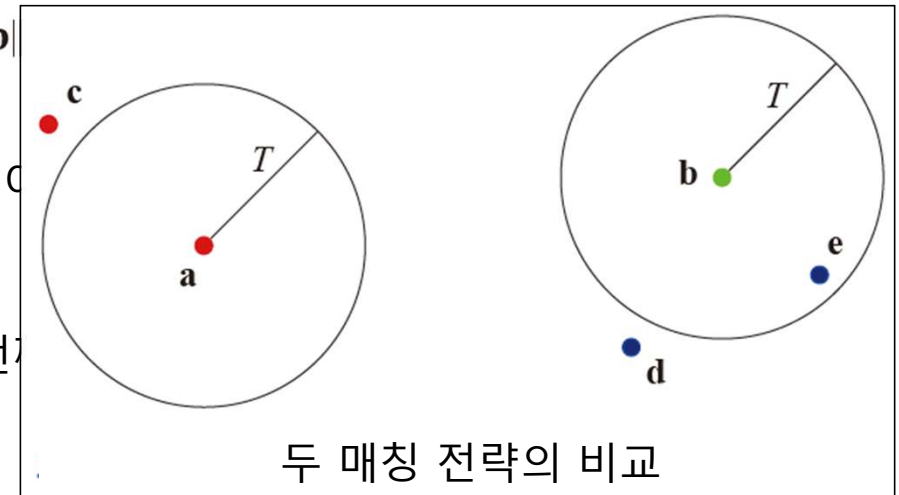
- 매칭 전략

- 최근접 이웃: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 를 찾고 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 가

$$d(\mathbf{a}_i, \mathbf{b}_j) < T \quad T: \text{임계값}$$

- 최근접 이웃 거리 비율: $\mathbf{a}_i$ 는 B 에서 가장 가까운 $\mathbf{b}_j$ 와 두 번만족시키면 $\mathbf{a}_i$ 와 $\mathbf{b}_j$ 는 매칭쌍으로 간주

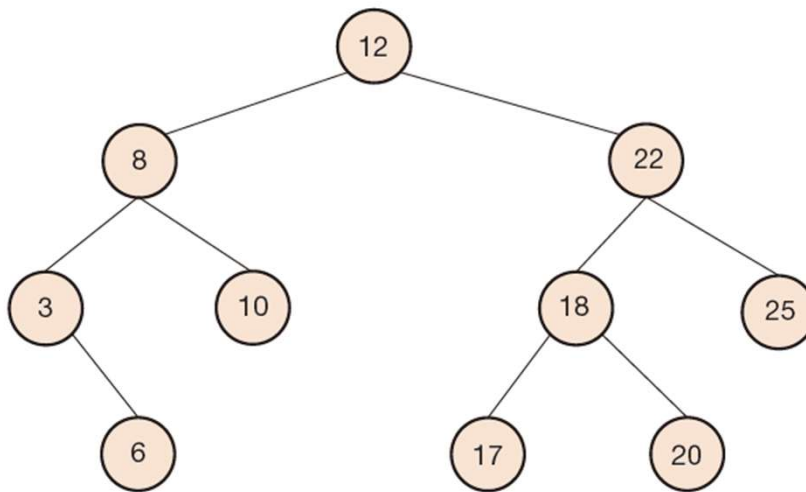
$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$



SIFT도 최근접 이웃 거리 비율 전략을 사용
(Lowe's Ratio Test)

빠른 매칭

- 빠른 탐색을 위한 자료 구조를 도입해 사용
 - 이진 탐색 트리(Binary search tree)와 해싱(Hashing) 등



이진 탐색 트리(Binary search tree)

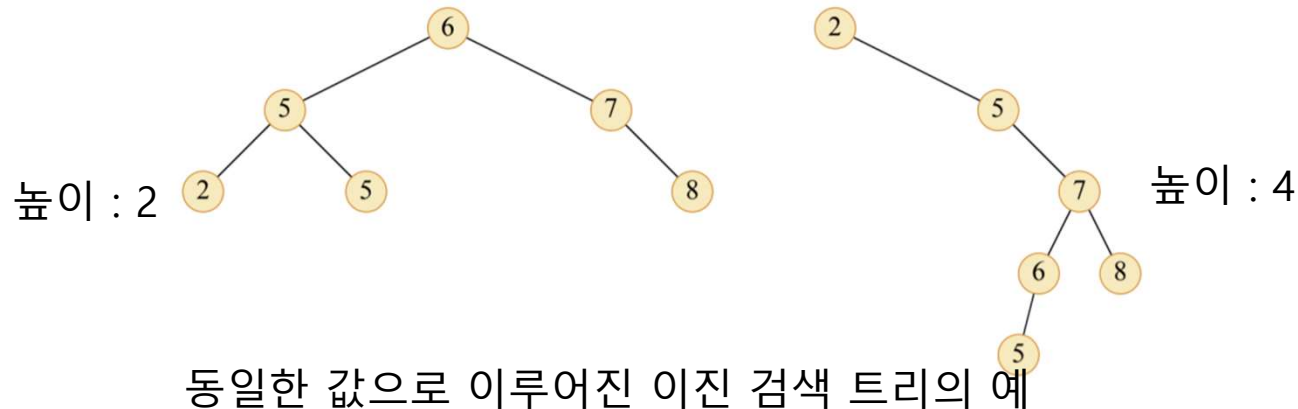
해시 함수
 $h(x) = x \% 13$

0	
1	14
2	
3	
4	134
5	
6	
7	7
8	
9	
10	65023
11	
12	

해싱(Hashing)

빠른 매칭

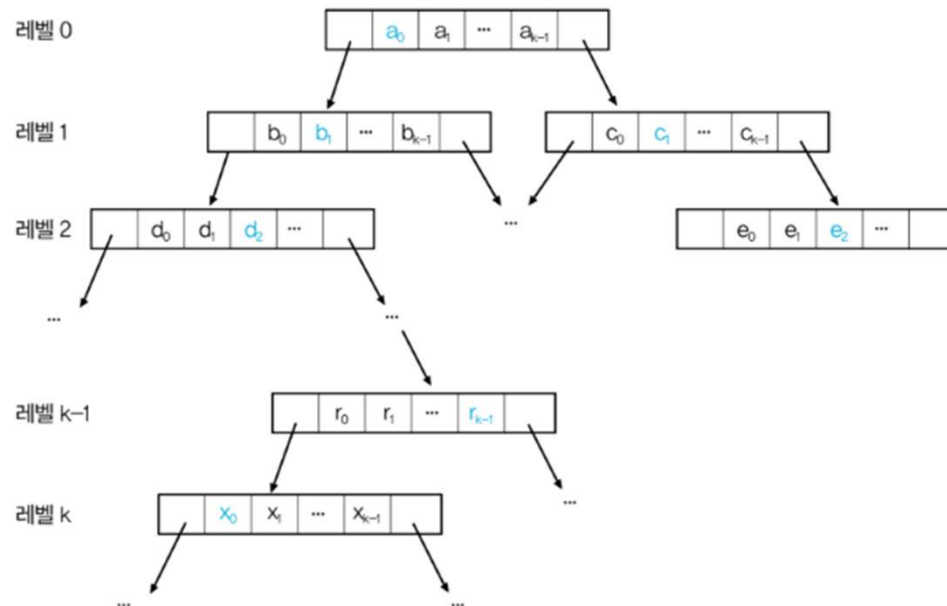
- 검색 트리(Search tree)
 - 데이터를 트리 구조로 저장하여 빠른 탐색 및 저장 등이 가능하도록 만든 자료 구조
- 이진 탐색 트리(Binary search tree)
 - 최상위 레벨에 루트 노드가 있고 한 노드에서 최대 2개의 자식 노드를 가짐
 - 이진 검색 트리에 대한 기본 연산은 트리의 높이에 비례하는 시간에 수행
 - 임의의 노드의 값은 자신의 왼쪽에 있는 모든 노드의 값보다 크고 오른쪽에 있는 모든 노드의 값보다 작음
 - 값들의 삽입 순서에 따라 모양이 결정



빠른 매칭

- kd 트리

- 특징의 기술자(descriptor)는 벡터이기 때문에 이진 탐색 트리 그대로 적용이 불가능
- k 차원 공간의 점들을 효율적으로 저장하고 탐색하기 위한 이진 공간 분할(binary space partitioning) 트리
- 이진 검색 트리를 확장한 것으로 k 개($k \geq 2$)의 필드(차원)로 이루어지는 벡터를 사용
- 검색 트리의 각 레벨이 하나씩의 차원만 다름

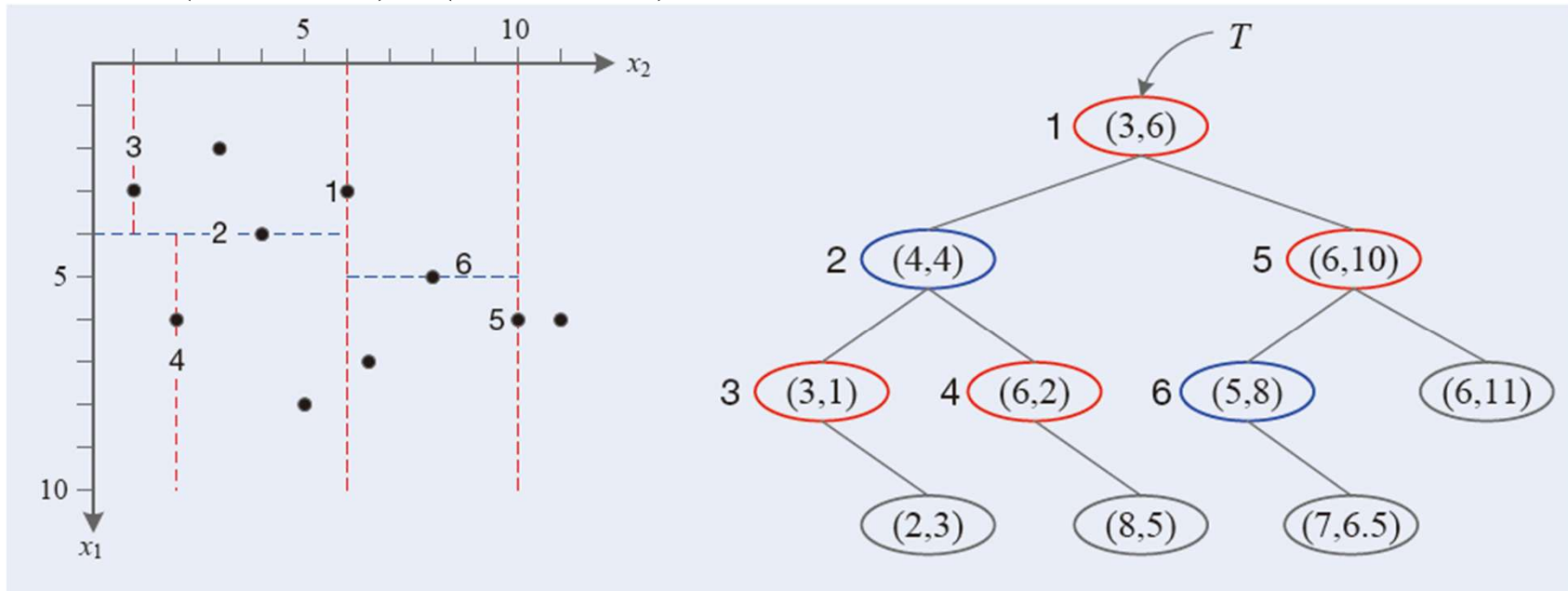


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 두 축 값은 (3,2,6,4,...,6)과 (1,3,2,4,...,11). 두 번째 축의 분산이 더 커서 두 번째 축 선택

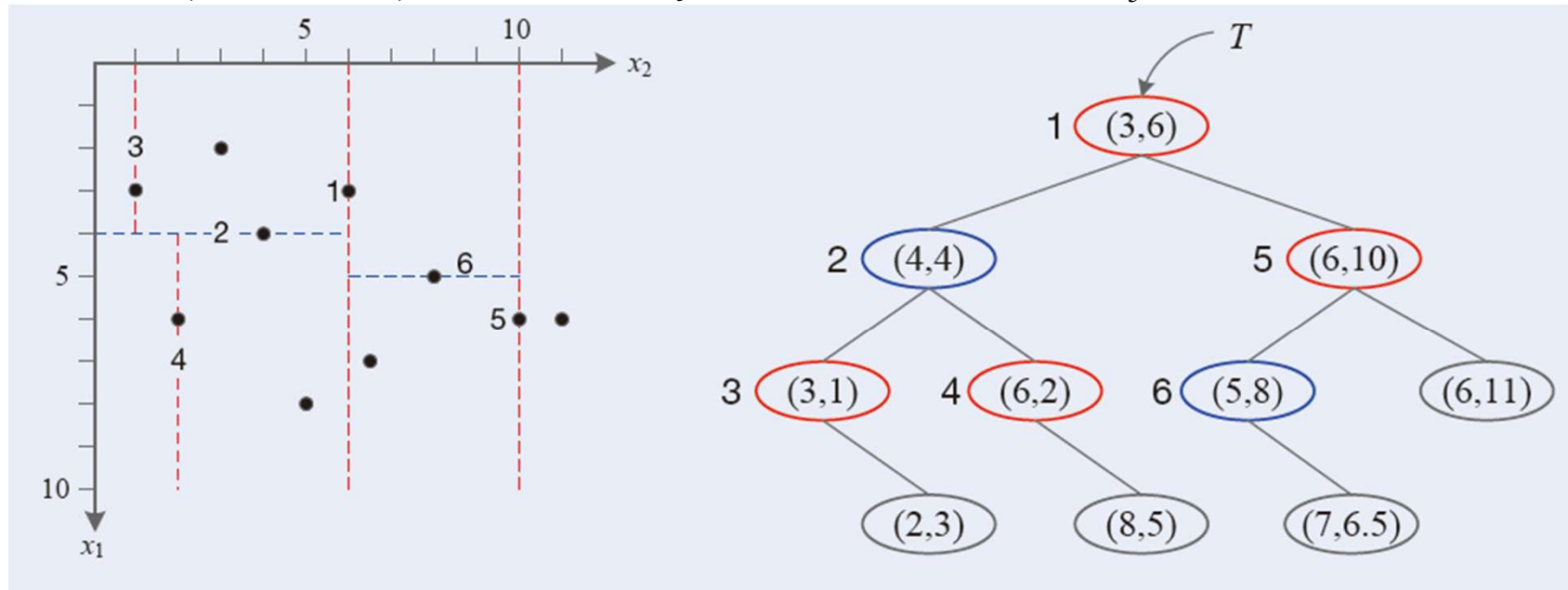


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 두 번째 축 (1,3,2,4,...,11) 정렬. 중앙값은 $j=5$, 즉 다섯 번째 기술자 x_5 가 분할 기준

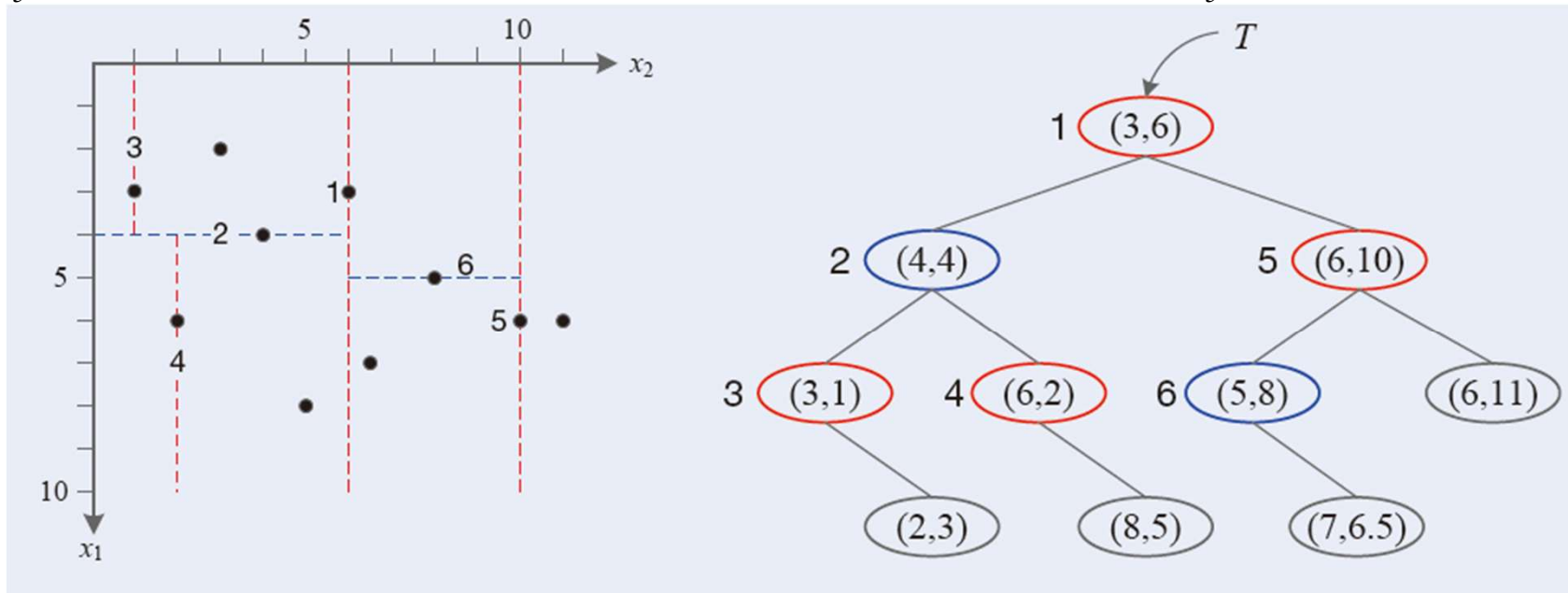


빠른 매칭

- kd 트리 만들기

$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- x_5 와 두 번째 축을 기준으로 나머지 9개 기술자를 분할 및 루트 노드에 x_5 를 배치하고 분할 정보(축) 저장

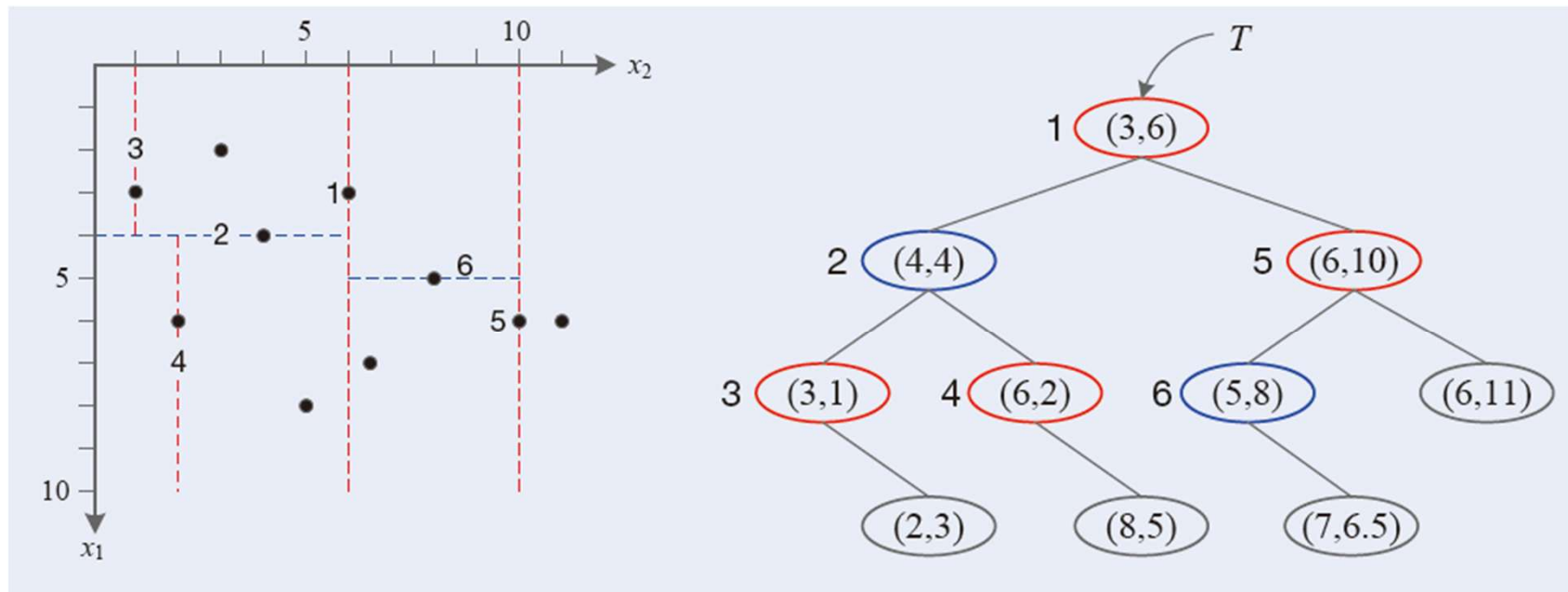


빠른 매칭

- kd 트리 만들기

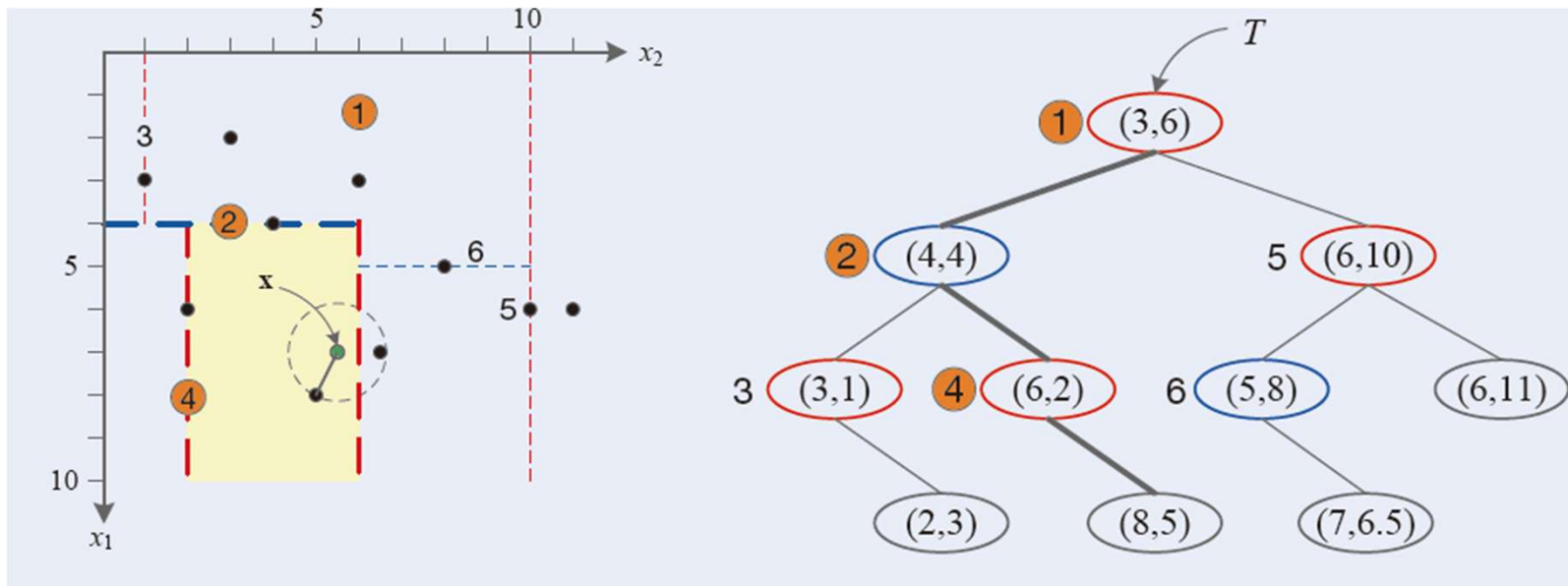
$X = \{x_1 = (3,1), x_2 = (2,3), x_3 = (6,2), x_4 = (4,4), x_5 = (3,6), x_6 = (8,5), x_7 = (7,6.5), x_8 = (5,8), x_9 = (6,10), x_{10} = (6,11)\}$

- 왼쪽 및 오른쪽 기술자들에 대해 같은 과정을 재귀적으로 반복



빠른 매칭

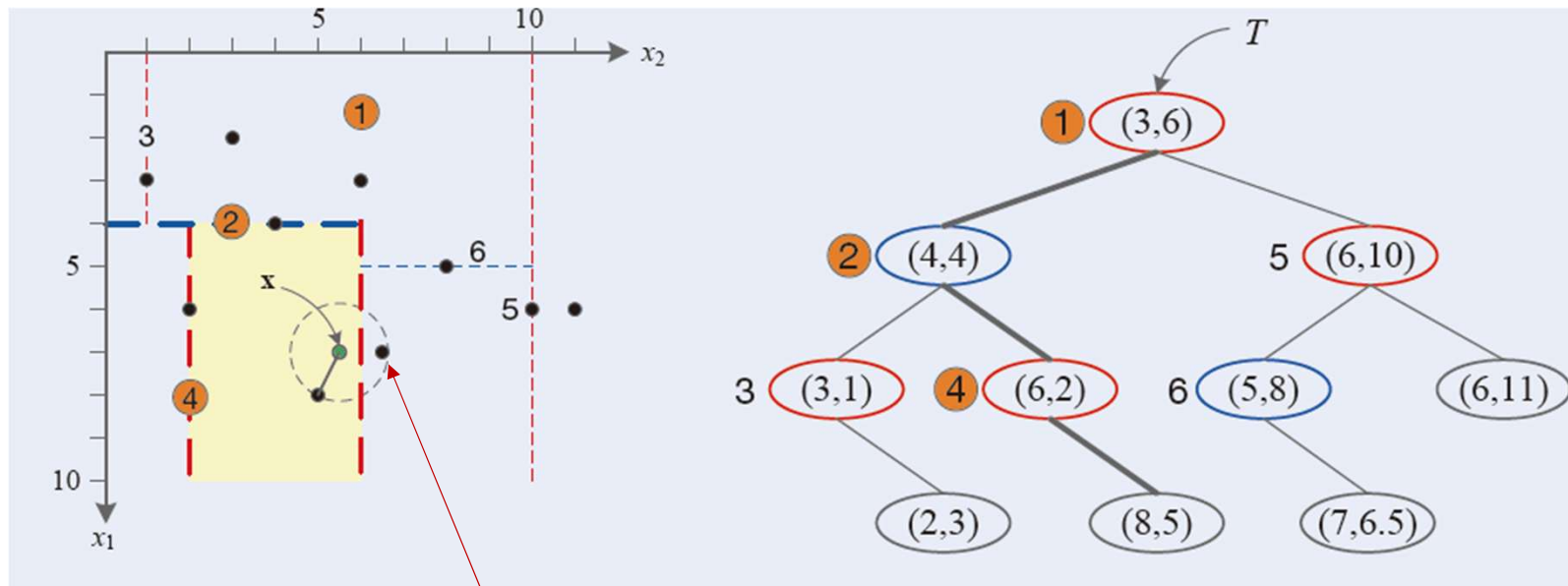
- kd 트리를 이용한 최근접 탐색
 - 새로운 특징 벡터 $\mathbf{x}=(7, 5.5)$ 가 입력되었다고 가정하고 최근접 이웃을 찾는 과정



kd 트리 탐색

빠른 매칭

- kd 트리를 이용한 최근접 탐색
 - 새로운 특징 벡터 $\mathbf{x}=(7, 5.5)$ 가 입력되었다고 가정하고 최근접 이웃을 찾는 과정



kd 트리 탐색

더 가까운 점 -> 타고 내려온 경로를 역추적하여 더 가까운 점을 찾는 추가 과정 필요

빠른 매칭

- FLANN(Fast Library for Approximate Nearest Neighbors)
 - 근사 최근접 이웃(Approximate Nearest Neighbor, ANN) 탐색 라이브러리
 - 사용자가 최적의 알고리즘을 고를 필요 없이 데이터의 특성에 맞춰 자동으로 가장 빠른 탐색 알고리즘(kd Tree, KMeans Tree 등)을 선택해 주는 기능
 - OpenCV에 내장되어 있어 접근성이 높음
 - 설정이 간편하고 SIFT 같은 전통적인 영상 간 특징점 매칭에 최적화

SIFT 실습

```
import cv2
import numpy as np
import urllib.request
from google.colab.patches import cv2_imshow

# 이미지 다운로드
url =
'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg'
urllib.request.urlretrieve(url, 'butterfly.jpg')
img = cv2.imread('butterfly.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# 비교를 위해 이미지를 강제로 회전 및 축소시킴
rows, cols = gray.shape
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 0.7) # 45도 회전, 0.7배 축소
img_varied = cv2.warpAffine(gray, M, (cols, rows))

# SIFT 객체 생성 및 특징점(Keypoints) 검출
sift = cv2.SIFT_create()
```

```
# kp: 특징점 위치/크기 정보, des: 특징점을 설명하는 128차원 벡터(Descriptor)
kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(img_varied, None)

# 특징점 매칭 (FLANN Matcher 사용)
# trees=5: 5개의 무작위 KD-트리를 병렬로 만들어 탐색 속도를 높임
# checks=50: 트리를 얼마나 깊게 뒤져볼지 결정
FLANN_INDEX_KDTREE = 1
index_params = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)

flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2) # 가장 가까운 2개 특징 찾기

# 좋은 매칭점만 선별 (Lowe's ratio test)
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)
```

SIFT 실습

```
# 결과 시각화
# 두 이미지 사이의 매칭 라인을 그림
img_match = cv2.drawMatches(gray, kp1, img_varied, kp2, good_matches, None,
                             flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)

print(f'검출된 특징점 개수: {len(kp1)}')
print(f'매칭 성공 개수: {len(good_matches)}')
cv2.imshow(img_match)
```

중간고사 일정

- 컴퓨터 비전 1, 2반 중간고사 일정(잠정)
 - 4월 23일 목요일
 - 오후 4시 30분~6시
 - 총점: 40점

매칭 성능 측정

- 정확도(Accuracy)와 F1-Score

- 같은 색이 실제 매칭쌍(정답, GT)이고 연결선이 매칭 알고리즘이 맺어준 쌍(예측)이라고 했을 때



- 혼동 행렬(confusion matrix)

		정답(GT)	
		긍정	부정
예측	긍정	참 긍정(TP)	거짓 긍정(FP)
	부정	거짓 부정(FN)	참 부정(TN)

정확도 (Accuracy)

- 전체 데이터 중 정답으로 분류되는 비율

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

매칭 성능 측정

- 정확도(Accuracy)와 F1-Score

- 같은 색이 실제 매칭쌍(정답, GT)이고 연결선이 매칭 알고리즘이 맺어준 쌍(예측)이라고 했을 때



- 혼동 행렬(confusion matrix)

		정답(GT)	
		긍정	부정
예측	긍정	참 긍정(TP)	거짓 긍정(FP)
	부정	거짓 부정(FN)	참 부정(TN)

정밀도 (Precision)

- 매칭으로 예측했을 때 실제로 매칭인 비율

$$Precision = \frac{TP}{TP+FP}$$

재현율 (Recall, Sensitivity)

- 실제 매칭들 중 매칭으로 예측되는 비율

$$Recall = \frac{TP}{TP+FN}$$

F1-Score (F-Score)

- 정밀도와 재현율의 조화평균

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

매칭 성능 측정

- 정확도(Accuracy)와 F1-Score 측정 예시
 - 혼동 행렬(confusion matrix)

		정답(GT)	
		긍정	부정
예측	긍정	참 긍정(TP) = 20	거짓 긍정(FP) = 10
	부정	거짓 부정(FN) = 10	참 부정(TN) = 60

정확도 (Accuracy)

- 전체 데이터 중 정답으로 분류되는 비율
- $$Accuracy = \frac{TP+TN}{TP+TN+FP+F} = \frac{20+60}{100} = 0.8$$

정밀도 (Precision)

- 매칭으로 예측했을 때 실제로 매칭인 비율

$$Precision = \frac{TP}{TP+FP} = \frac{20}{20+10} = 0.666 \dots$$

재현율 (Recall, Sensitivity)

- 실제 매칭들 중 매칭으로 예측되는 비율

$$Recall = \frac{TP}{TP+FN} = \frac{20}{20+10} = 0.666 \dots$$

F1-Score (F-Score)

- 정밀도와 재현율의 조화평균

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision+Recall} = 2 \cdot \frac{0.667 \cdot 0.667}{0.667 + 0.667} = 0.667$$

매칭 성능 측정

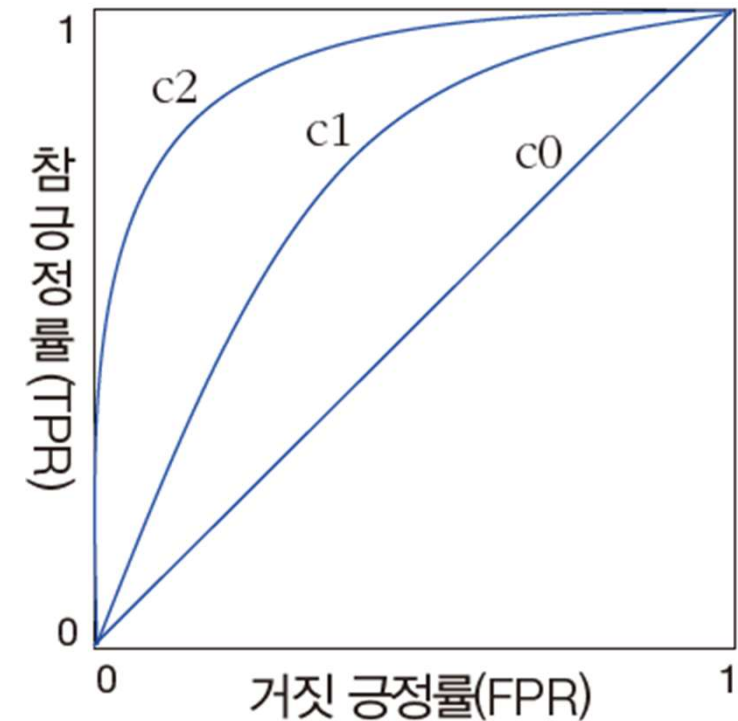
- FPS (Frames Per Second)
 - 1초당 처리할 수 있는 프레임 수
 - 시스템이 얼마나 부드럽게 실시간으로 작동하는지를 나타냄
 - 계산: $1000 / \text{Latency (ms)}$
- 레이턴시 (Latency / Processing Time)
 - 한 프레임의 데이터가 입력되어 결과가 나올 때까지 걸리는 전체 지연 시간
 1. Detection Time: 특징 위치를 찾는 시간 (예: FAST)
 2. Description Time: 특징을 이진/실수 형태의 기술자로 변환하는 시간 (예: BRIEF)
 3. Matching Time: 두 기술자 사이의 거리를 계산하는 시간 (예: Hamming)
- 처리량 (Throughput)
 - 단위 시간당 얼마나 많은 특징점이나 데이터를 처리했는지를 나타냄
 - 단위: Points per second (Pts/s)
 - 하드웨어(CPU/GPU)의 자원을 얼마나 효율적으로 쥐어짜내고 있는지 보여줌. 예를 들어, 멀티코어를 사용하여 병렬 처리를 하면 레이턴시는 그대로여도 처리량은 늘어날 수 있음

매칭 성능 측정

- ROC(Receiver Operating Characteristic) 곡선
 - 아래 식에서 T 를 작게 하면 거짓 긍정률(FPR, False Positive Rate)은 작아짐
 - T 를 크게 하면 거짓 긍정률이 커지는데 참 긍정률 (TPR, True Positive Rate) 도 따라 커지는 경향이 있음

$$\frac{d(\mathbf{a}_i, \mathbf{b}_j)}{d(\mathbf{a}_i, \mathbf{b}_k)} < T \quad T: \text{임계값}$$

$$\left. \begin{aligned} \text{참 긍정률} &= \frac{TP}{TP + FN} \\ \text{거짓 긍정률} &= \frac{FP}{TN + FP} \end{aligned} \right\} \quad (5.17)$$



c2가 가장 좋은 알고리즘

ORB(Oriented FAST and Rotated BRIEF)

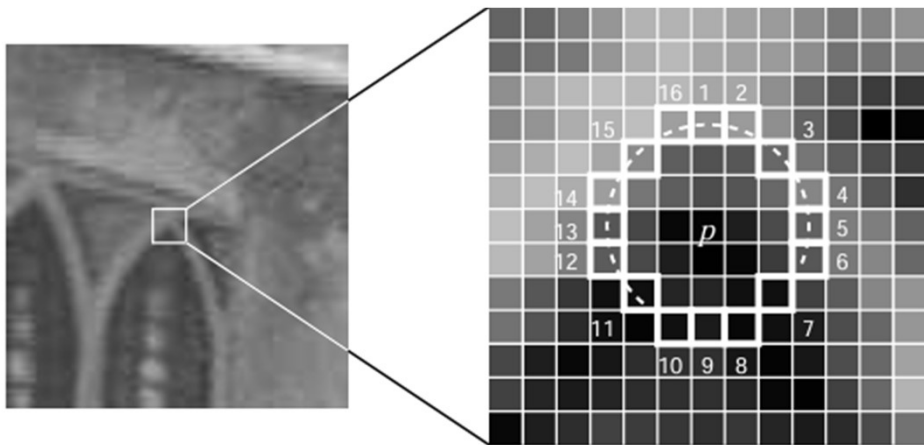
- ORB(Oriented FAST and Rotated BRIEF)
 - SIFT 등 기존 특징 검출 및 기술 알고리즘의 단점(특허+느린속도) 보완
 - 실시간성이 필요한 분야(AR, SLAM 등)에서 많이 사용
 - 특징 검출(feature detection)
 - Oriented + FAST
 - 기존 FAST: 주변 픽셀과의 밝기 차이만 비교하여 점을 찾기 때문에 매우 빠르지만, 이미지가 회전하면 특징점을 놓치는 치명적인 단점이 있음
 - 특징점의 기하학적 중심에서 밝기 중심까지의 벡터를 구해 방향(Orientation) 정보를 추가
- 특징 기술(feature description)
 - Rotated + BRIEF
 - 기존 BRIEF: 특징점 주변 픽셀 쌍의 밝기를 비교해 0 또는 1로 기록하는 이진 벡터 방식
 - 계산은 빠르지만 회전에 매우 취약
 - ORB의 개선: 특징 검출 단계에서 구한 방향 정보를 이용해 픽셀 쌍의 위치를 함께 회전(Steered BRIEF)시켜 계산
 - 픽셀 쌍 중 변별력이 가장 높은 쌍들만 골라내는 최적화 과정을 거침

ORB(Oriented FAST and Rotated BRIEF)

- ORB(Oriented FAST and Rotated BRIEF)

- 특징 검출(feature detection)

- Oriented + FAST(Features from Accelerated Segment Test)



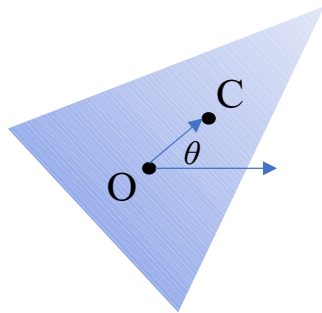
FAST(Features from Accelerated Segment Test)

- 픽셀 p 를 중심으로 상하좌우, 대각선 방향의 총 16개의 픽셀들과 값을 비교
- 차이가 n 개 이상(9개) 연속으로 존재하면 특징점으로 판단

- 기존 FAST: 주변 픽셀과의 밝기 차이만 비교하여 점을 찾기 때문에 매우 빠르지만 영상이 회전하면 특징점을 놓치는 단점이 있음

ORB(Oriented FAST and Rotated BRIEF)

- ORB(Oriented FAST and Rotated BRIEF)
 - 특징 검출(feature detection)
 - Oriented + FAST(Features from Accelerated Segment Test)



기하학적 중심(O)으로부터 밝기 중심(C)으로의 방향이 Orientation(θ)을 나타냄

- 기존 FAST: 주변 픽셀과의 밝기 차이만 비교하여 점을 찾기 때문에 매우 빠르지만 영상이 회전하면 특징점을 놓치는 단점이 있음
- 특징점의 기하학적 중심에서 밝기 중심까지의 벡터를 구해 방향(Orientation) 정보를 추가

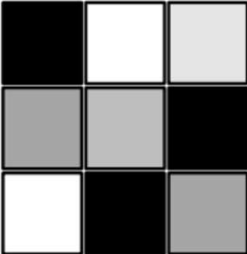
ORB(Oriented FAST and Rotated BRIEF)

- ORB(Oriented FAST and Rotated BRIEF)

- 특징 기술(feature description)

- Rotated + BRIEF(Binary Robust Independent Elementary Features)

- BRIEF: 특징점 주변 픽셀 쌍의 밝기를 비교해 0 또는 1로 기록하는 이진 벡터 방식

image area	index (for pairs)		
	1	2	3
	4	5	6
	7	8	9

- 키 포인트 주변의 랜덤 픽셀 쌍을 선택
 - 첫번째 픽셀이 두번째 픽셀보다 밝다면 '1'을 부여하고 반대라면 '0'
 - 예) pairs: {(5,1), (5,9), (4,6), (8,2), (3,7)}
 - tests: b=0 b=0 b=0 b=1 b=1
 - result: B=00011

- 특징점 주변의 정보를 256 bits로 표현

- ORB의 개선: 특징 검출 단계에서 구한 방향 정보를 이용해 픽셀 쌍의 위치를 함께 회전(Steered BRIEF)시켜 계산

- 픽셀 쌍 중 변별력이 가장 높은(uncorrelated and high variant) 쌍들만 골라내는 최적화 과정을 거침

ORB(Oriented FAST and Rotated BRIEF)

- ORB(Oriented FAST and Rotated BRIEF)
 - 특징 기술(feature description)
 - Rotated + BRIEF(Binary Robust Independent Elementary Features)
 - BRIEF: 특징점 주변 픽셀 쌍의 밝기를 비교해 0 또는 1로 기록하는 이진 벡터 방식
 - 특징점 주변의 정보를 256 bits로 표현
 - 특징들 간의 거리는 Hamming distance로 계산(XOR 연산으로 매우 빠르게 계산)
 - Hamming distance
 - 아래와 같이 두 개의 8비트 데이터가 있을 때

A: 1 0 1 1 1 0 1 0

B: 1 0 0 1 0 0 1 1

XOR: 0 0 1 0 1 0 0 1 → Hamming distance : 3

ORB(Oriented FAST and Rotated BRIEF)

- SIFT와 ORB의 비교

비교 항목	SIFT	ORB
속도	상대적으로 느림	매우 빠름 (실시간 처리용)
데이터 형태	128차원 실수 벡터	256비트 이진 벡터 (0과 1)
매칭 방식	Euclidean distance	Hamming distance (XOR 연산)
라이선스	과거 특허(현재 만료)	완전 무료 (BSD)

ORB 실습

```
import time

import numpy as np

import urllib.request

from google.colab.patches import cv2_imshow # 코랩 전용 출력

# 이미지 로드 및 흑백 변환

url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
urllib.request.urlretrieve(url, 'aloeL.jpg')

img1 = cv2.imread('aloeL.jpg', cv2.IMREAD_GRAYSCALE)

url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'
urllib.request.urlretrieve(url, 'aloeR.jpg')

img2 = cv2.imread('aloeR.jpg', cv2.IMREAD_GRAYSCALE)

def benchmark_full(name, detector, matcher_type='BF', ratio_threshold=0.7):
    start_time = time.time()

    kp1, des1 = detector.detectAndCompute(img1, None)
    kp2, des2 = detector.detectAndCompute(img2, None)

    detect_time = (time.time() - start_time) * 1000
```

```
# 매처(Matcher) 설정

if matcher_type == 'BF':
    norm = cv2.NORM_L2 if name == "SIFT" else cv2.NORM_HAMMING
    matcher = cv2.BFMatcher(norm, crossCheck=False)
else:
    if name == "SIFT":
        index_params = dict(algorithm=1, trees=5) # FLANN_INDEX_KDTREE
    else:
        index_params = dict(algorithm=6, table_number=6, key_size=12,
                              multi_probe_level=1)

    search_params = dict(checks=50)

    matcher = cv2.FlannBasedMatcher(index_params, search_params)

# k-NN 매칭 및 Ratio Test

start_time = time.time()

matches = matcher.knnMatch(des1, des2, k=2)
```

ORB 실습

```
good_matches = []

for m_n in matches:
    if len(m_n) == 2:
        m, n = m_n
        if m.distance < ratio_threshold * n.distance:
            good_matches.append(m)

match_time = (time.time() - start_time) * 1000

print(f"\n--- [{name} + {matcher_type}] ---")
print(f"Detect: {detect_time:.1f}ms | Match: {match_time:.1f}ms | Goods: {len(good_matches)}")

# 매칭 결과 출력 코드
# 상위 50개(혹은 전체)만 그림
res_img = cv2.drawMatches(img1, kp1, img2, kp2, good_matches[:50], None,
                           flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

```
# 결과 이미지 위에 텍스트 정보 표시
info_text = f'{name}+{matcher_type}: {len(good_matches)} matches'
cv2.putText(res_img, info_text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

cv2.imshow(res_img)

# -----
return kp1, kp2, good_matches

# 벤치마크 실행
sift = cv2.SIFT_create()
orb = cv2.ORB_create(nfeatures=2000)

print("SIFT 벤치마크 시작...")
benchmark_full("SIFT", sift, "BF")
benchmark_full("SIFT", sift, "FLANN")

print("\nORB 벤치마크 시작...")
benchmark_full("ORB", orb, "BF", ratio_threshold=0.85)
benchmark_full("ORB", orb, "FLANN", ratio_threshold=0.85)
```

ORB 실습

- ORB의 사용 분야
 - 특히 실시간성이 생명인 분야에서 많이 쓰임
 - SLAM (자율주행/로봇): 로봇이 움직이며 초당 수십 프레임의 영상을 처리해 위치를 잡아야 할 때 (예: ORB-SLAM3).
 - 모바일 AR (증강현실): 스마트폰의 제한된 연산 자능으로 즉시 가상 물체를 합성해야 할 때.

ORB(Oriented FAST and Rotated BRIEF)

- 위치 의존 해싱(Locality Sensitive Hashing, LSH)
 - 일반적인 해싱
 - 해시 함수로 $O(1)$ 시간에 탐색 달성
 - 효율적인 해시 함수와 충돌 해결 기법이 개발되어 있음
 - 특징점 매칭에 사용하려면 개조 필요
 - 특징 벡터이고 최근접 이웃 찾아야 하기 때문
 - 충돌을 최소화하는 일반 해싱과 달리 비슷한 특징 벡터를 같은 칸에 담아야 함
 - 위치 의존 해싱은 이런 조건을 만족하는 방법

해시 함수

$$h(x) = x \% 13$$

0	
1	14
2	
3	
4	134
5	
6	
7	7
8	
9	
10	65023
11	
12	

해싱(Hashing)

ORB 실습

```
import cv2
import numpy as np
import time
from google.colab.patches import cv2_imshow

# 샘플 영상 로드
url = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/vtest.avi'
urllib.request.urlretrieve(url, 'vtest.avi')
cap = cv2.VideoCapture('vtest.avi')

# 타겟 설정 (첫 프레임의 특정 영역)
ret, first_frame = cap.read()
if not ret:
    exit()

target_gray = cv2.cvtColor(first_frame, cv2.COLOR_BGR2GRAY)
target_roi = target_gray[100:300, 200:400] # 보행자가 있는 영역 절삭
```

```
# ORB 및 매처 초기화
orb = cv2.ORB_create(nfeatures=1000)
kp1, des1 = orb.detectAndCompute(target_roi, None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)

print(f"{'Frame':<8} | {'Detect(ms)':<12} | {'Match(ms)':<12} | {'Total(ms)':<12} | {'FPS':<6}")
print("-" * 65)

frame_count = 0
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break
    # --- 속도 측정 시작 ---
    start_total = time.time()
    # 특징점 검출 및 기술자 생성 시간 측정
    start_det = time.time()
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    kp2, des2 = orb.detectAndCompute(gray_frame, None)
    end_det = time.time()
```

ORB 실습

```
det_time = (end_det - start_det) * 1000 # ms 단위
```

```
match_time = 0
```

```
if des2 is not None:
```

```
    # 매칭 시간 측정
```

```
    start_match = time.time()
```

```
    matches = bf.knnMatch(des1, des2, k=2)
```

```
    good_matches = [m for m, n in matches if m.distance < 0.75 * n.distance]
```

```
    end_match = time.time()
```

```
    match_time = (end_match - start_match) * 1000
```

```
# --- 속도 측정 종료 ---
```

```
end_total = time.time()
```

```
total_time = (end_total - start_total) * 1000
```

```
fps = 1.0 / (end_total - start_total) if total_time > 0 else 0
```

```
# 지표 출력 (매 10프레임마다)
```

```
if frame_count % 10 == 0:
```

```
    print(f'{frame_count:<8} | {det_time:<12.2f} | {match_time:<12.2f} |  
{total_time:<12.2f} | {fps:<6.1f}')
```

```
# 시각화 (매 100프레임마다)
```

```
if frame_count % 100 == 0:
```

```
    res = cv2.drawMatches(target_roi, kp1, frame, kp2, good_matches[:20], None,
```

```
                           flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

```
    cv2.putText(res, f'FPS: {fps:.1f}', (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1,  
(0, 255, 0), 2)
```

```
    cv2.imshow(res)
```

```
frame_count += 1
```

Similarity Search

- Faiss(Facebook AI Similarity Search)
 - 고차원 벡터의 유사도 검색(Similarity Search) 및 클러스터링을 위한 오픈소스 라이브러리
 - 대규모 벡터에서 빠르게 유사한 데이터를 찾는데 사용
 - 주요 특징
 - 근사 최근접 이웃(ANN, Approximate Nearest Neighbor) 탐색을 위한 다양한 알고리즘 지원
 - GPU 가속: CUDA를 활용하여 수십억 개의 연산을 병렬로 처리합니다.
 - 압축 (Quantization): 벡터의 용량을 1/10 이하로 줄여 메모리 효율을 극대화
 - 적용 분야
 - 대규모의 이미지를 검색하거나 실시간으로 엄청난 양의 특징점을 처리해야 하는 경우
 - 추천 시스템(예)수천만 명의 사용자 취향 벡터 중 나와 비슷한 유저 찾기)
 - 영상이나 문장을 벡터로 변환하여 내용 기반 검색 수행

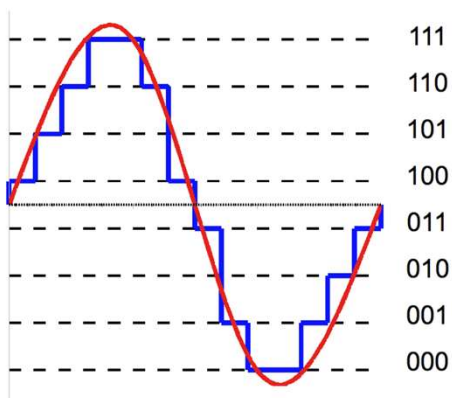
Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Faiss의 주요 알고리즘

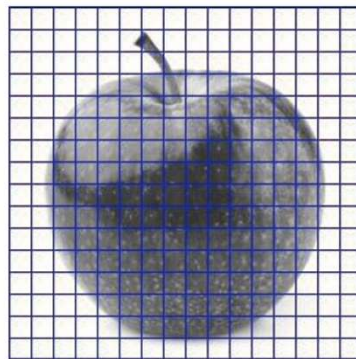
인덱스 명칭	특징	추천 상황
IndexFlatL2	전수 조사 (Exact Search). 가장 정확함	데이터가 적고(100만 개 이하) 정확도가 중요할 때
IndexIVFFlat	데이터를 여러 구역(Cell)으로 나누어 검색 범위를 제한	속도와 정확도의 균형이 필요할 때
IndexPQ	Product Quantization. 데이터를 압축하여 메모리 절약	대규모의 데이터를 메모리에 올려야 할 때
IndexHNSW	그래프 기반 탐색. 현존하는 ANN 중 가장 빠르고 정확함	메모리 여유가 있고 초고속 검색이 필요할 때

Similarity Search

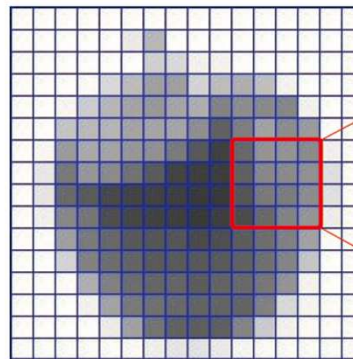
- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - 연속적이거나 고차원의 값을 대표값으로 근사하는 것
 - Scalar quantization(스칼라 양자화)



양자화(3 bits)

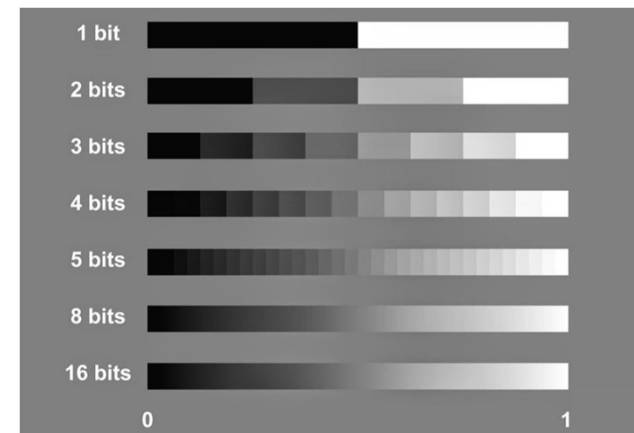


샘플링



양자화(8 bits)

105	149	149	141
97	137	139	146
86	123	126	142
76	106	132	150



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Vector quantization(벡터 양자화)
 1. 코드북(Codebook) 생성: 데이터 공간을 여러 구역으로 나누고 각 구역의 중심점(Centroid)을 정함. 중심점들의 모음이 코드북이 됨
 2. 인코딩(Encoding): 입력 벡터가 들어오면, 코드북에서 가장 가까운 중심점을 찾음
 3. 치환: 원래의 복잡한 벡터 데이터를 보내는 대신 그 중심점의 인덱스만 저장하거나 전송



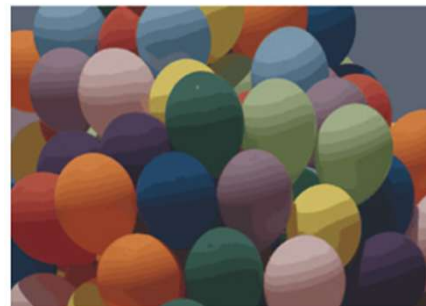
원본 영상



$k = 4$



$k = 16$



$k = 64$



$k = 256$

영상의 vector quantization 적용 예시

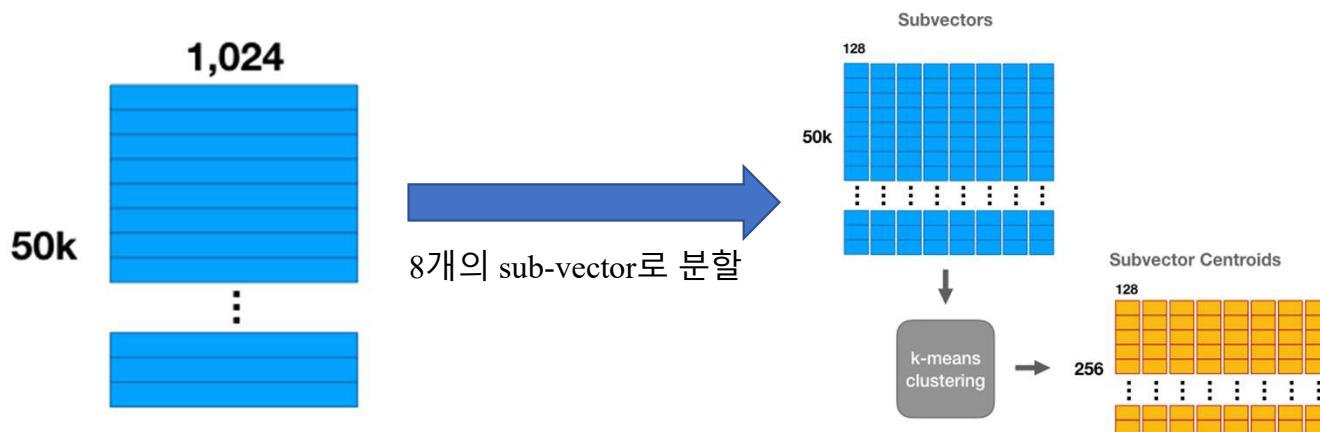
Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - 아주 긴 벡터를 여러 개의 짧은 조각으로 나눈 뒤 각 조각을 미리 정해둔 코드북으로 대체하는 방법
 - 일반적으로 원래의 데이터를 수십배 이상 줄일 수 있음
 - 예) 128차원의 SIFT 특징점 10억 개 : $128 \times 4 \times 10^9 \approx 512 \text{ GB}$
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈

Similarity Search

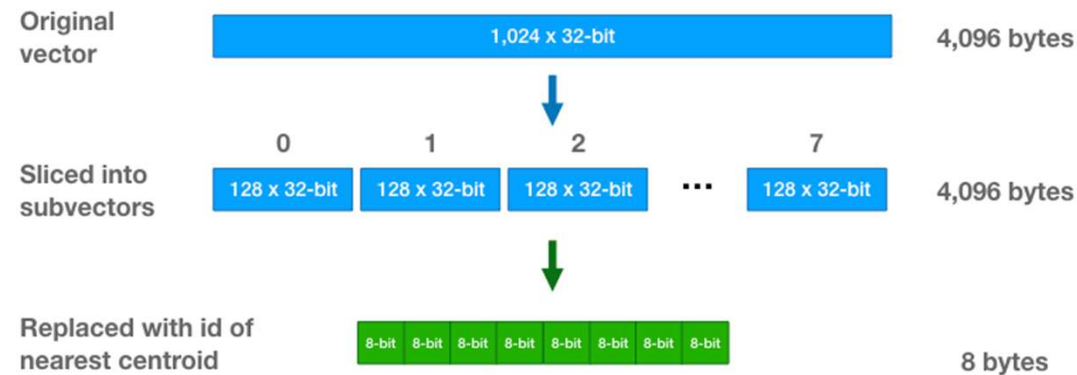
- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈

- 50000개의 특징들
- 각 특징마다 1024차원



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - PQ 과정
 1. 하나의 긴 벡터를 동일한 크기의 작은 조각(Sub-vector) m개로 나눔
 2. 학습 데이터를 이용해 각 조각들의 Centroid들을 K-평균(K-means) 클러스터링으로 찾아냄
 3. 실제 데이터를 저장할 때 각 조각을 가장 닮은 centroid의 인덱스로 바꿈



Similarity Search

- Faiss(Facebook AI Similarity Search)
 - Product quantization
 - 고차원 벡터를 여러 부분으로 나눠 각각 양자화(quantization)하여 압축하는 방법
 - 양자화(quantization)
 - Product quantization
 - 거리 계산
 - Query vector의 각 조각(sub-vector)과 db의 각 조각 내 클러스터들 각각의 centroid에 대해 L2 거리를 계산하여 테이블을 구축
 - 예) Query vector의 조각의 개수가 8개이고 db의 각 조각마다 256개의 centroid를 가진다고 하면 테이블의 크기는 $256 \times 8 = 2048$
 - db에 저장된 데이터와 거리를 비교 시 데이터의 각 조각에서의 인덱스에 해당하는 거리를 테이블에서 가져와서 모두 더함
 - 예) db의 데이터의 수가 50000개이고 그 중 하나의 데이터가 [12, 105, 3, 210, 45, 90, 17, 200]라고 하면
 1. 테이블의 첫번째 조각의 12번째 인덱스에 해당하는 거리값을 가져옴.
 2. 이를 여덟번째 조각까지 찾고 모두 더함

Similarity Search 실습

1. 코랩 GPU 설정

[수정] -> [노트북 설정] -> [하드웨어 가속기] -> T4 GPU로 변경

2. 설치

```
!pip install faiss-gpu-cu12
```

3. 설치 확인

```
import faiss
print(faiss.__version__)
print(faiss.get_num_gpus())
```

4. 코드 실행

```
import cv2
import numpy as np
import faiss
import time
import urllib.request
from google.colab.patches import cv2_imshow

# GPU 리소스 초기화
res = faiss.StandardGpuResources()
```

이미지 로드 및 특징점 추출 (데이터(특징점)의 수: 10,000개)

```
url1 = '
https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg '
url2 = '
https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg '

Urllib.request.urlretrieve(url1, 'aloeL.jpg')
Urllib.request.urlretrieve(url2, 'aloeR.jpg')
```

```
img1 = cv2.imread('aloeL.jpg', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('aloeR.jpg', cv2.IMREAD_GRAYSCALE)
```

```
sift = cv2.SIFT_create(nfeatures=10000)
kp1, des1 = sift.detectAndCompute(img1, None)
kp2, des2 = sift.detectAndCompute(img2, None)
```

```
des1_f32 = des1.astype('float32')
des2_f32 = des2.astype('float32')
```

```
print(f"특징점 개수: Image1({len(kp1)}), Image2({len(kp2)})\n")
```

Similarity Search 실습

```
def run_benchmark_and_show(name, search_func, is_faiss=False):
    # --- 순수 속도 측정 (시각화 제외) ---
    search_func() # 워밍업 (GPU 메모리 할당 및 초기화 방지)

    start_time = time.time()
    for _ in range(10):
        raw_result = search_func()
    end_time = time.time()
    avg_ms = ((end_time - start_time) / 10) * 1000
    print(f"[{name:<18}] 순수 매칭 속도: {avg_ms:>7.2f} ms")
    # --- [B] 시각화를 위한 후처리 (측정 범위 밖) ---
    good_matches = []
    if not is_faiss: # OpenCV Matcher (BF, FLANN)
        for m, n in raw_result:
            if m.distance < 0.7 * n.distance:
                good_matches.append(m)
    else: # Faiss (Flat, PQ)
        D, I = raw_result
        for i in range(len(des1_f32)):
            if D[i][0] < 0.7 * D[i][1]:
                m = cv2.DMatch(_queryIdx=i, _trainIdx=int(I[i][0]), _distance=D[i][0])
                good_matches.append(m)
```

```
# 매칭 결과 그리기

res_img = cv2.drawMatches(img1, kp1, img2, kp2, good_matches[:1000], None,
                           flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
cv2.imshow(res_img)
print(f'추출된 유효 매칭 수: {len(good_matches)}\n' + "-"*80)

# OpenCV Brute-Force (CPU)
bf = cv2.BFMatcher(cv2.NORM_L2)
run_benchmark_and_show("BF-Matcher (CPU)", lambda: bf.knnMatch(des1, des2,
k=2))

# OpenCV FLANN (CPU)
flann = cv2.FlannBasedMatcher(dict(algorithm=1, trees=5), dict(checks=50))
run_benchmark_and_show("FLANN (CPU)", lambda: flann.knnMatch(des1, des2, k=2))

# Faiss FlatL2 (GPU)
index_flat = faiss.IndexFlatL2(128)
gpu_index_flat = faiss.index_cpu_to_gpu(res, 0, index_flat)
gpu_index_flat.add(des2_f32)

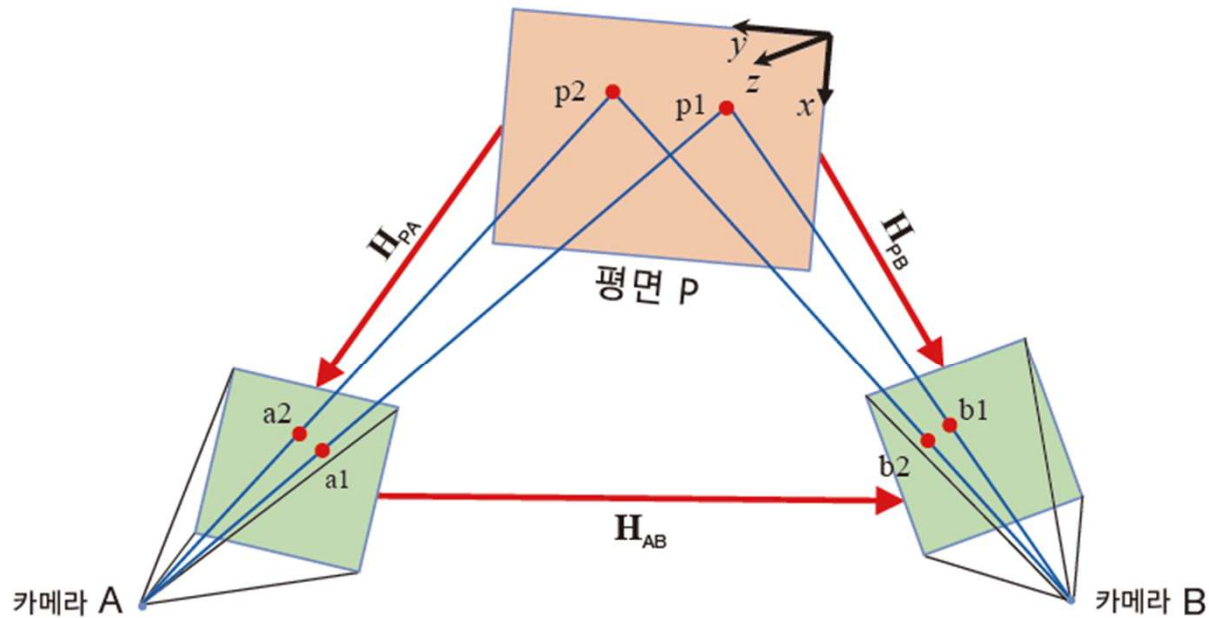
run_benchmark_and_show("Faiss Flat (GPU)", lambda: gpu_index_flat.search(des1_f32,
2), is_faiss=True)
```

Similarity Search 실습

```
# Faiss IVF-PQ (GPU)
# 128차원을 16개 조각(m=16)으로 나누어 8비트(nbits=8)로 양자화
nlist = 100
m = 16
quantizer = faiss.IndexFlatL2(128)
index_pq = faiss.IndexIVFPQ(quantizer, 128, nlist, m, 8)
gpu_index_pq = faiss.index_cpu_to_gpu(res, 0, index_pq)
gpu_index_pq.train(des2_f32)
gpu_index_pq.add(des2_f32)
run_benchmark_and_show("Faiss IVF-PQ (GPU)", lambda: gpu_index_pq.search(
des1_f32, 2), is_faiss=True)
```

Homography(호모그래피) 추정

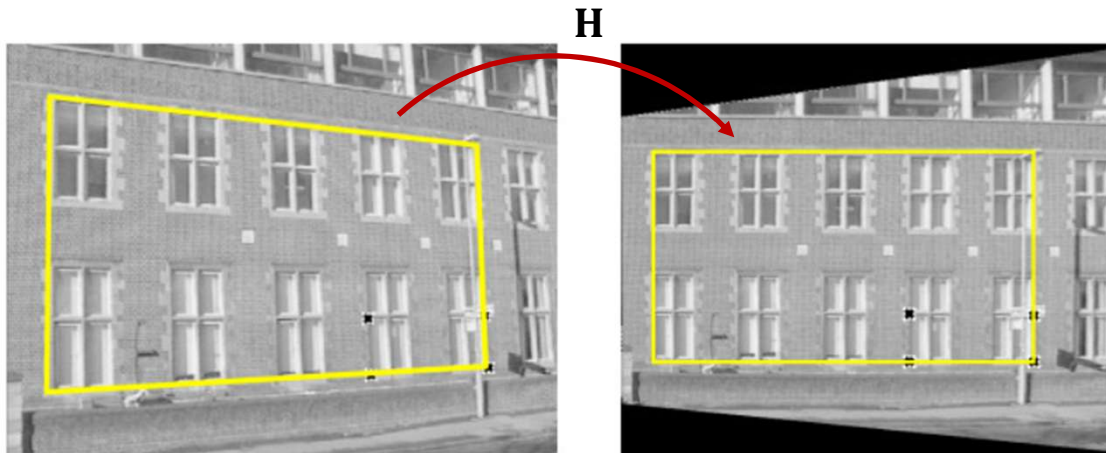
- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현



Homography(호모그래피)

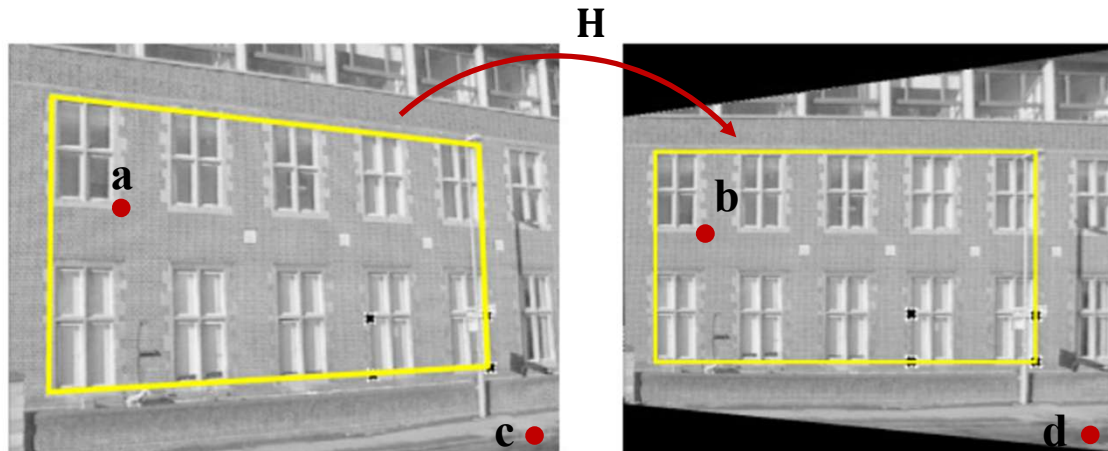
Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3×3 행렬 H 로 표현



Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 H 로 표현



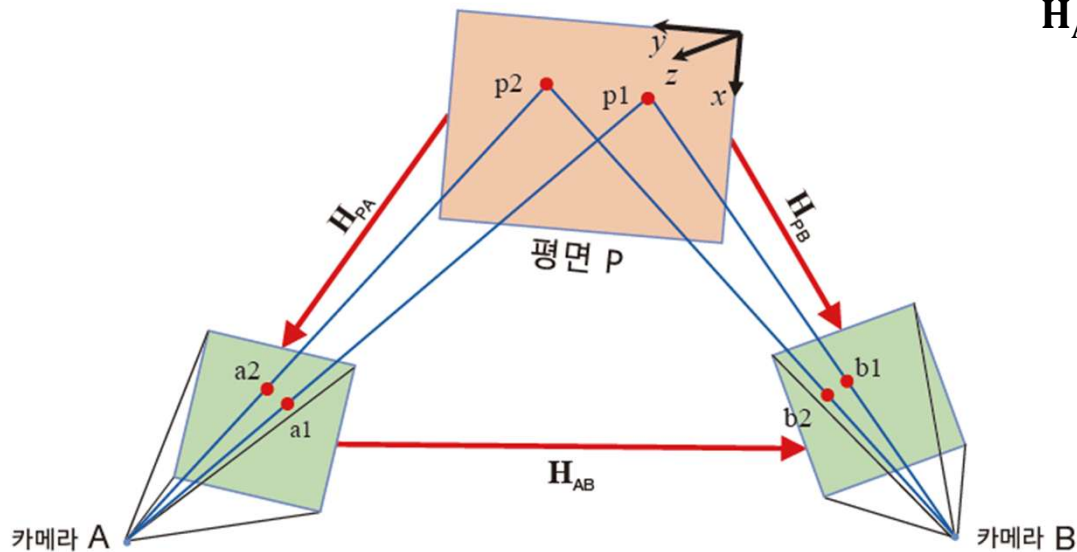
$$Ha = b$$

$$Hc \neq d$$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현

$$\mathbf{H}_{AB} = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$$



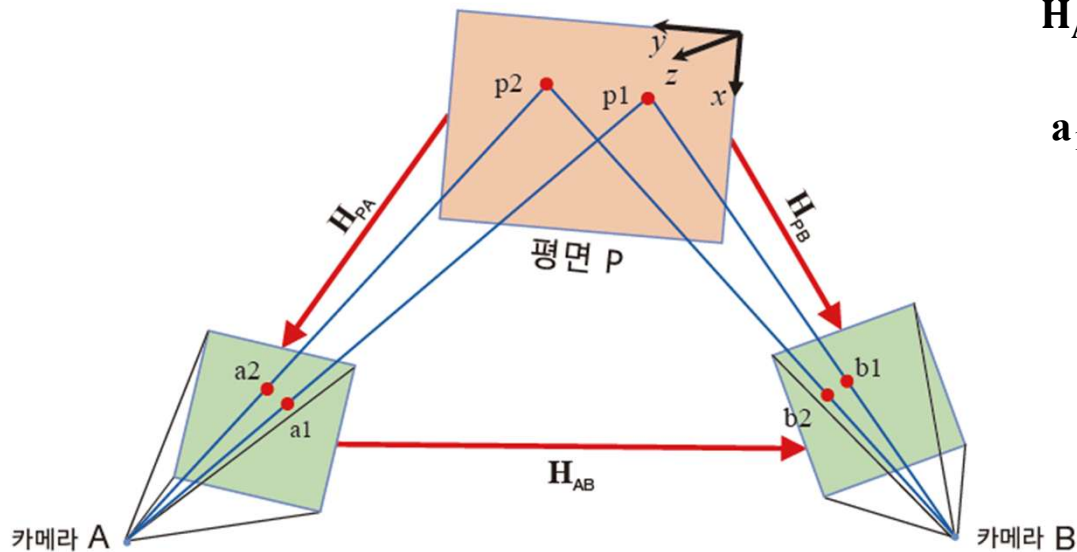
$$\mathbf{H}_{AB}\mathbf{a}_1 = \mathbf{b}_1$$

$$\mathbf{H}_{AB}\mathbf{a}_2 = \mathbf{b}_2$$

Homography(호모그래피)

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현



Homography(호모그래피)

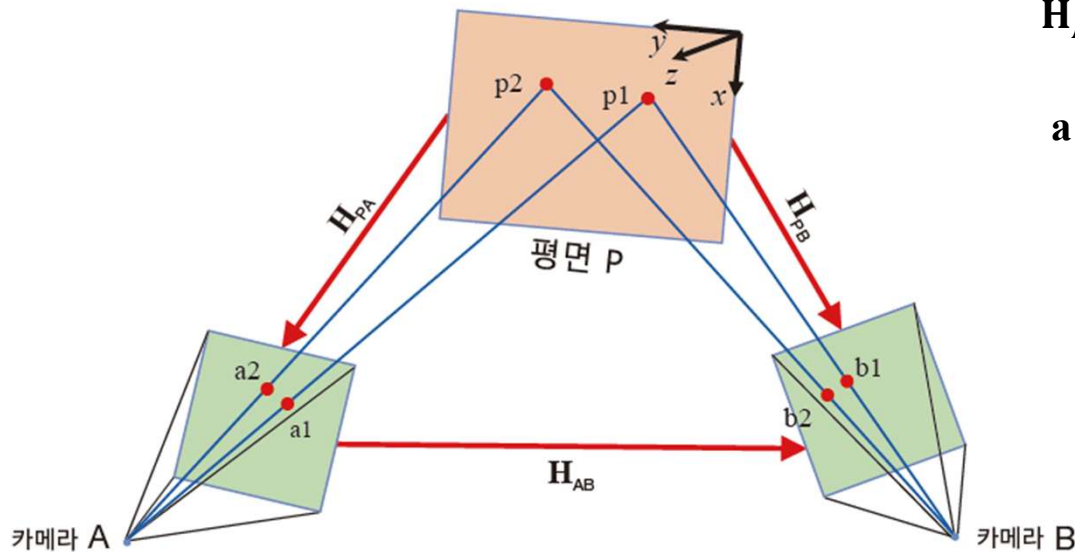
$$\mathbf{H}_{AB} = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$$

$\mathbf{a}_1 = (x_{a1}, y_{a1}, 1)^T$, $\mathbf{b}_1 = (x_{b1}, y_{b1}, 1)^T$ 이라고 하면

$$\begin{aligned} \mathbf{H}_{AB}\mathbf{a}_1 &= \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix} \begin{bmatrix} x_{a1} \\ y_{a1} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} h_{11}x_{a1} + h_{21}y_{a1} + h_{31} \\ h_{12}x_{a1} + h_{22}y_{a1} + h_{32} \\ h_{13}x_{a1} + h_{23}y_{a1} + h_{33} \end{bmatrix} = \mathbf{b}_1 \end{aligned}$$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현



Homography(호모그래피)

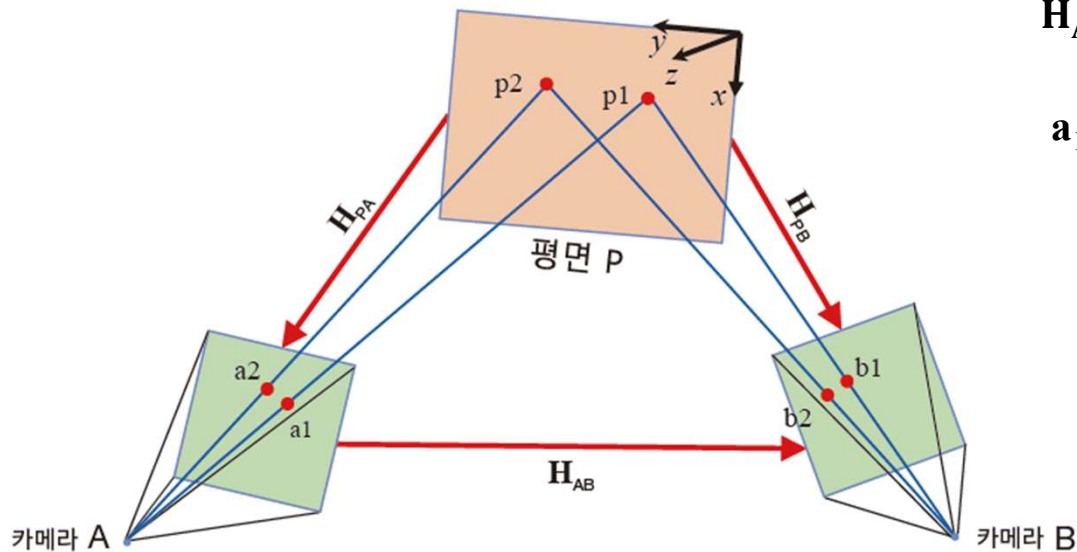
$$\mathbf{H}_{AB} = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$$

$\mathbf{a}_1 = (x_{a1}, y_{a1}, 1)^T$, $\mathbf{b}_1 = (x_{b1}, y_{b1}, 1)^T$ 이라고 하면

$$\begin{aligned} \mathbf{H}_{AB}\mathbf{a}_1 &= \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix} \begin{bmatrix} x_{a1} \\ y_{a1} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} h_{11}x_{a1} + h_{21}y_{a1} + h_{31} \\ h_{12}x_{a1} + h_{22}y_{a1} + h_{32} \\ h_{13}x_{a1} + h_{23}y_{a1} + h_{33} \end{bmatrix} = \mathbf{b}_1 \\ &= \begin{bmatrix} \frac{h_{11}x_{a1} + h_{21}y_{a1} + h_{31}}{h_{13}x_{a1} + h_{23}y_{a1} + h_{33}} \\ \frac{h_{12}x_{a1} + h_{22}y_{a1} + h_{32}}{h_{13}x_{a1} + h_{23}y_{a1} + h_{33}} \\ 1 \end{bmatrix} = \mathbf{b}_1 \quad (\text{homogeneous coordinates}) \end{aligned}$$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현



Homography(호모그래피)

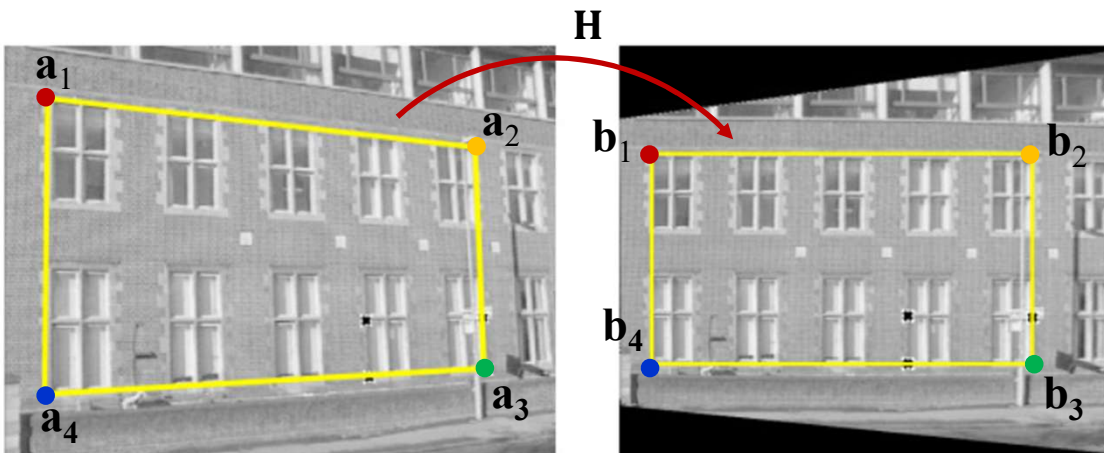
$$\mathbf{H}_{AB} = \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$$

$\mathbf{a}_1 = (x_{a1}, y_{a1}, 1)^T$, $\mathbf{b}_1 = (x_{b1}, y_{b1}, 1)^T$ 이라고 하면

$$\begin{aligned} \mathbf{H}_{AB}\mathbf{a}_1 &= \begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix} \begin{bmatrix} x_{a1} \\ y_{a1} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} h_{11}x_{a1} + h_{21}y_{a1} + h_{31} \\ h_{12}x_{a1} + h_{22}y_{a1} + h_{32} \\ h_{13}x_{a1} + h_{23}y_{a1} + h_{33} \\ 1 \end{bmatrix} = \mathbf{b}_1 \text{ (homogeneous coordinates)} \\ x_{b1} &= \frac{h_{11}x_{a1} + h_{21}y_{a1} + h_{31}}{h_{13}x_{a1} + h_{23}y_{a1} + h_{33}} & y_{b1} &= \frac{h_{12}x_{a1} + h_{22}y_{a1} + h_{32}}{h_{13}x_{a1} + h_{23}y_{a1} + h_{33}} \end{aligned}$$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현
 - Homography를 결정하는 최소한의 매칭점들(matching points, correspondences)의 수는 4개임



$\mathbf{a}_1 = (x_{a1}, y_{a1}, 1)^T$, $\mathbf{b}_1 = (x_{b1}, y_{b1}, 1)^T$ 이라고 하면

$$\mathbf{b}_1 = \mathbf{H}\mathbf{a}_1$$

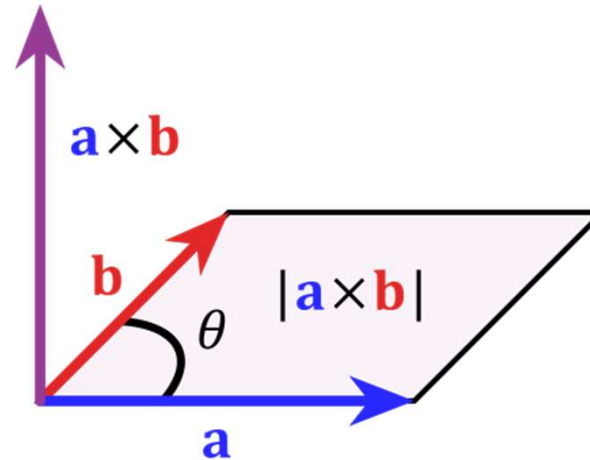
$$\text{즉 } \|\mathbf{b}_1 - \mathbf{H}\mathbf{a}_1\| = 0$$



$$\underline{\mathbf{b}_1 \times \mathbf{H}\mathbf{a}_1 = 0}$$

Homography(호모그래피) 추정

- 벡터(Vector)
 - 벡터의 외적(cross product)
 - $\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$
 - $|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta$



$$\mathbf{a} \times \mathbf{a} = \mathbf{0}$$

Homography(호모그래피) 추정

- Homography(호모그래피) 구하기

$\mathbf{a}_1 = (x_{a1}, y_{a1}, 1)^T$, $\mathbf{b}_1 = (x_{b1}, y_{b1}, 1)^T$ 이라고 하면

$$\mathbf{b}_1 \times \mathbf{H} \mathbf{a}_1 = 0$$

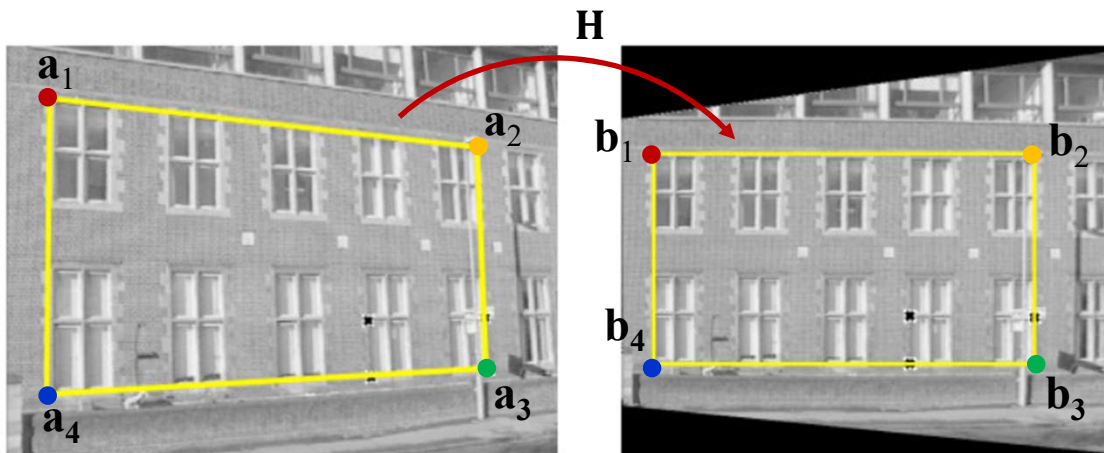
$$\Leftrightarrow \begin{bmatrix} x_{b1} \\ y_{b1} \\ 1 \end{bmatrix} \times \begin{bmatrix} h_{11}x_{a1} + h_{21}y_{a1} + h_{31} \\ h_{12}x_{a1} + h_{22}y_{a1} + h_{32} \\ h_{13}x_{a1} + h_{23}y_{a1} + h_{33} \end{bmatrix} = 0$$

$$\mathbf{a} \times \mathbf{b} = \begin{pmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{pmatrix}$$

$$\Leftrightarrow \underbrace{\begin{bmatrix} 0 & 0 & 0 & -x_{a1} & -y_{a1} & -1 & x_{a1}y_{b1} & y_{a1}y_{b1} & y_{b1} \\ x_{a1} & y_{a1} & 1 & 0 & 0 & 0 & -x_{a1}x_{b1} & -y_{a1}x_{b1} & -x_{b1} \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix}}_{\mathbf{h}} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} : \text{하나의 매칭점으로 2개의 방정식 얻음}$$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 H 로 표현
 - Homography를 결정하는 최소한의 매칭점들(matching points, correspondences)의 수는 4개임



일치점 4개를 안다면

$$b_1 = H a_1$$

$$b_2 = H a_2$$

$$b_3 = H a_3$$

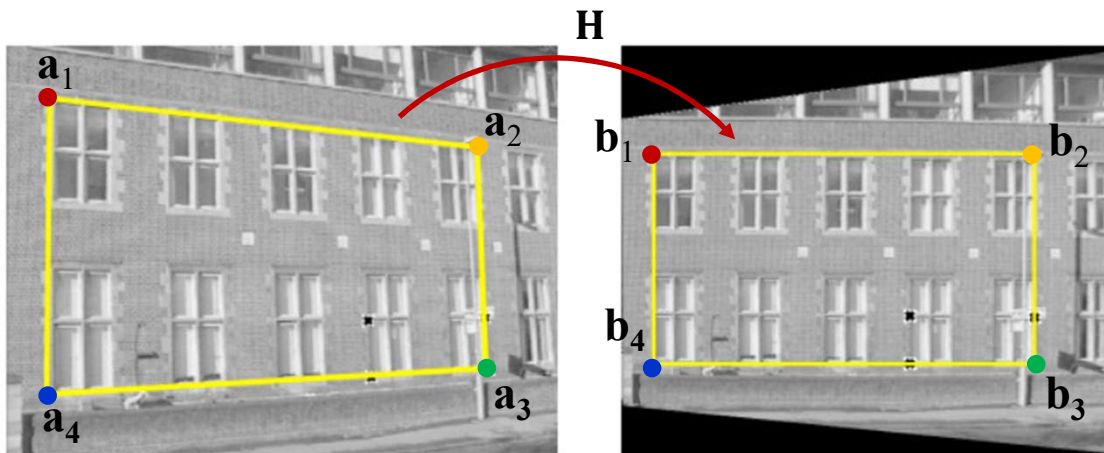
$$b_4 = H a_4$$

➡ 8개의 방정식

↓
 $Ah = 0$

Homography(호모그래피) 추정

- Homography(호모그래피)의 개념
 - 평면형 물체(planar object)를 카메라로 찍었을 때 원래의 물체와 영상 평면에 투영된 물체 사이에 성립되는 기하학적 변환 관계
 - homogeneous coordinates에서 3x3 행렬 $\mathbf{H}$ 로 표현
 - Homography를 결정하는 최소한의 매칭점들(matching points, correspondences)의 수는 4개임



일치점 4개를 안다면

$$\begin{aligned} \mathbf{b}_1 &= \mathbf{H}\mathbf{a}_1 \\ \mathbf{b}_2 &= \mathbf{H}\mathbf{a}_2 \\ \mathbf{b}_3 &= \mathbf{H}\mathbf{a}_3 \\ \mathbf{b}_4 &= \mathbf{H}\mathbf{a}_4 \end{aligned}$$

8개의 방정식

$$\mathbf{A}\mathbf{h} = \mathbf{0}$$

SVD(least squares)

$$\begin{bmatrix} h_{11} & h_{21} & h_{31} \\ h_{12} & h_{22} & h_{32} \\ h_{13} & h_{23} & h_{33} \end{bmatrix}$$

8개의 변수 결정
(h_{33} 는 scale factor)

Homography(호모그래피) 추정

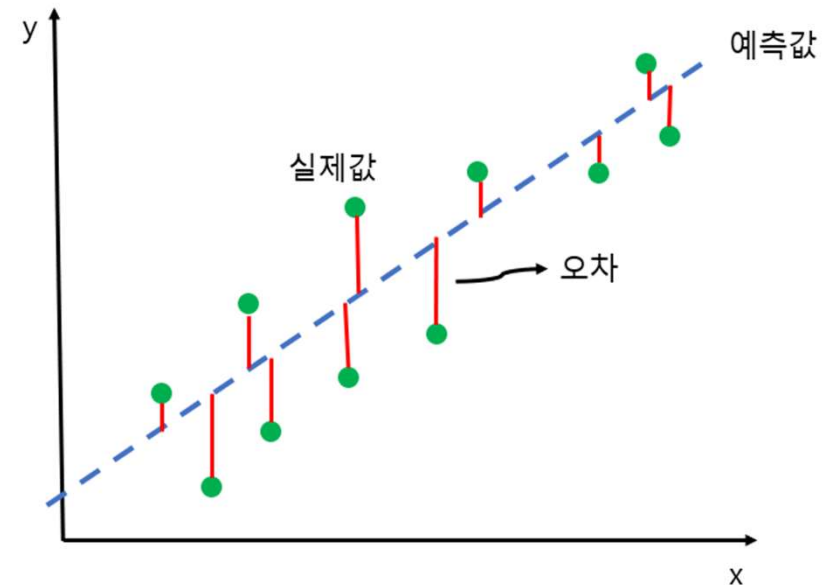
- Least squares(최소자승법)
 - 오차의 제곱합을 최소화하여 최적의 해를 찾는 방법

$$\min_x \|Ax - b\|^2$$

- 아래와 같은 명확한 해가 존재

$$x = (A^T A)^{-1} A^T b$$

- 수학적으로 미분/최적화가 편리함
- Outlier에 매우 취약
 - 오차²의 합들이라서 큰 오차(outlier)가 결과에 큰 영향을 미침



Homography(호모그래피) 추정

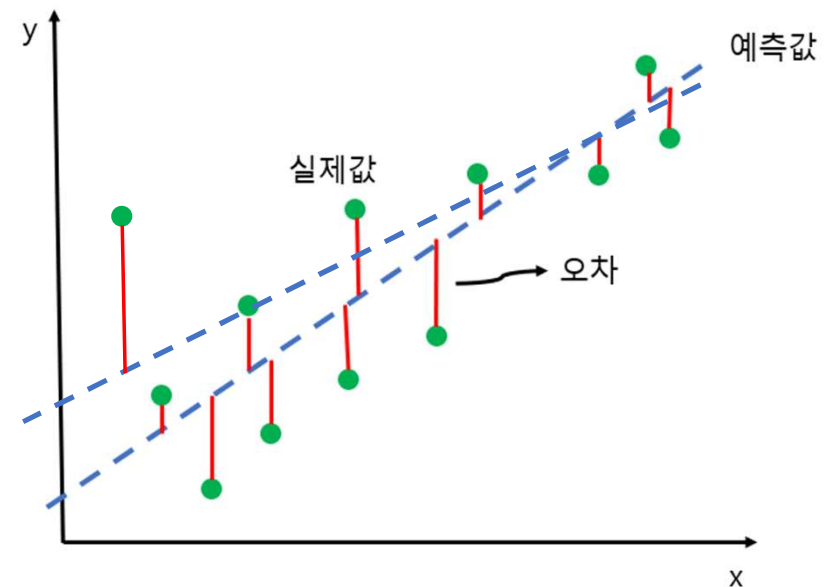
- Least squares(최소자승법)
 - 오차의 제곱합을 최소화하여 최적의 해를 찾는 방법

$$\min_x \|Ax - b\|^2$$

- 아래와 같은 명확한 해가 존재

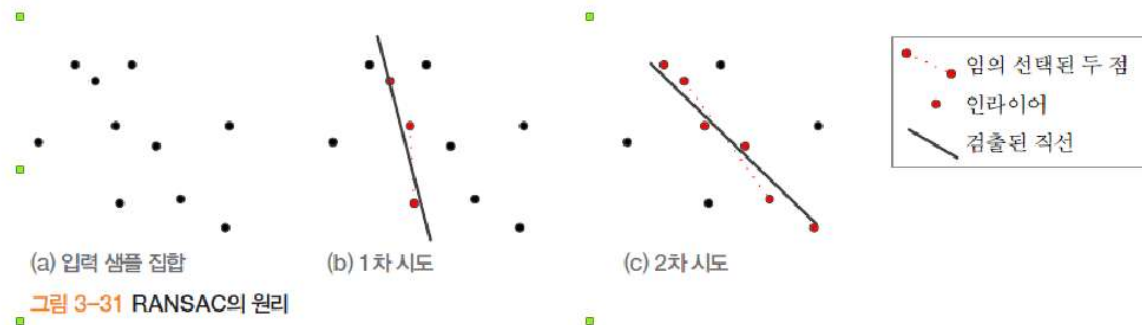
$$x = (A^T A)^{-1} A^T b$$

- 수학적으로 미분/최적화가 편리함
- Outlier에 매우 취약
 - 오차²의 합들이라서 큰 오차(outlier)가 결과에 큰 영향을 미침



RANSAC(RANdom Sample Consensus)

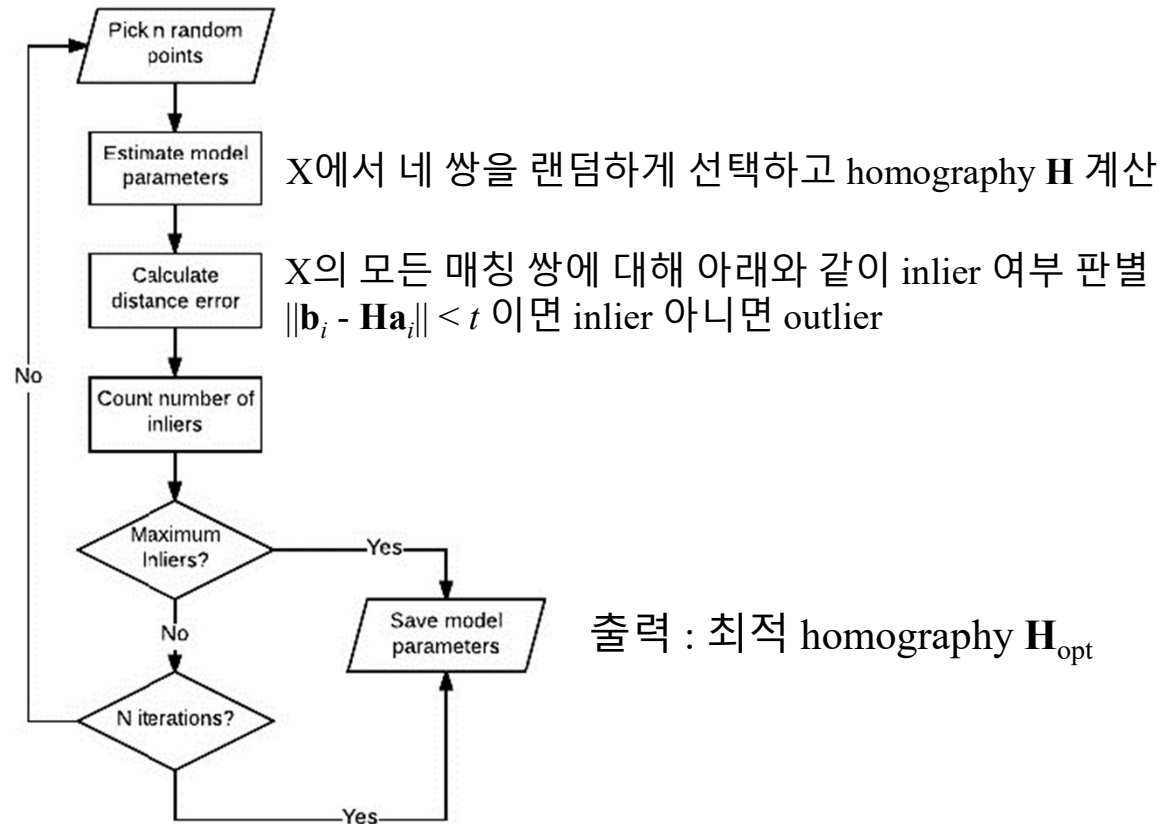
- 소수의 깨끗한 샘플만 무작위로 골라 가설을 세우고, 가장 많은 지지를 받는 가설을 선택하는 알고리즘
- 전체 순서
 1. 가설 설정 (Hypothesis): 데이터 중 아주 적은 수의 샘플(예: 직선의 경우 2개, homography의 경우 매칭 점 4개)을 무작위로 뽑음
 2. 모델 생성: 뽑은 샘플들로 임시 모델을 계산
 3. 검증 (Verification): 나머지 모든 데이터가 이 모델과 얼마나 가까운지 확인하고 정해진 오차 범위 내에 들어오면 인라이어(Inlier, 올바른 데이터)로 인정
 4. 최적 모델 선택: 위 과정을 여러 번 반복하여, 가장 많은 인라이어(Inlier)를 확보한 모델을 최종 정답으로 채택



RANSAC(RANdom Sample Consensus)

- Homography 추정을 위한 RANSAC

입력 : 매칭 쌍 집합 $X = \{(a_1, b_1), (a_2, b_2), \dots, (a_n, b_n)\}$



RANSAC(RANdom Sample Consensus)

- RANSAC 알고리즘의 특징
- 이상치(Outliers)에 매우 강함 (Robustness)
 - Least Squares은 모든 데이터의 오차 제곱합을 최소화하려다 보니 단 하나의 아주 큰 노이즈만 있어도 모델이 노이즈 쪽으로 크게 휘어버릴 수 있음
 - RANSAC은 전체 중 아주 일부만 무작위로 뽑아서 모델을 만든 뒤 얼마나 많은 데이터가 이 모델에 동의하는지(Consensus) 판단하기 때문에 데이터의 절반 이상이 Outliers여도 진짜 모델을 찾아낼 수 있음
- 특정 수학 모델에 국한되지 않고 다양한 기하학적 모델링에 모두 적용 가능
 - 예) 직선은 2개의 점, 평면은 3개의 점, homography는 4개의 매칭점 등 모델을 구성하는 최소한의 샘플을 선택할 수 있으면 적용 가능
- 샘플을 적게 뽑을수록 그 안에 오염된 데이터(Outlier)가 섞여 들어갈 확률이 적어지기 때문에 성공 확률과 연산 효율이 좋아짐
 - 예) 직선을 모델링할 때 3개의 점을 뽑을 때보다 2개의 점을 뽑을 때 outlier가 섞일 확률이 더 적음

RANSAC 실습

```
import cv2
import numpy as np
import faiss
import urllib.request
from google.colab.patches import cv2_imshow

# 1. GPU 리소스 초기화 및 이미지 로드
res = faiss.StandardGpuResources()

# 서로 다른 시점에서 촬영된 이미지 두 장
url1 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeL.jpg'
url2 = 'https://raw.githubusercontent.com/opencv/opencv/master/samples/data/aloeR.jpg'
urllib.request.urlretrieve(url1, 'aloeL.jpg')
urllib.request.urlretrieve(url2, 'aloeR.jpg')

img1_gray = cv2.imread('aloeL.jpg', cv2.IMREAD_GRAYSCALE)
img2_gray = cv2.imread('aloeR.jpg', cv2.IMREAD_GRAYSCALE)

# 2. SIFT 특징점 추출 (반복 패턴이라 특징점 개수를 충분히 늘림)
sift = cv2.SIFT_create(nfeatures=5000)
kp1, des1 = sift.detectAndCompute(img1_gray, None)
kp2, des2 = sift.detectAndCompute(img2_gray, None)
```

```
des1_f32 = des1.astype('float32')
des2_f32 = des2.astype('float32')

print(f"특징점 개수: Image1({len(kp1)}개), Image2({len(kp2)}개)\n")

# 3. Faiss-GPU를 이용한 고속 KNN 매칭 (K=2)
index = faiss.IndexFlatL2(128)
gpu_index = faiss.index_cpu_to_gpu(res, 0, index)
gpu_index.add(des2_f32)
distances, indices = gpu_index.search(des1_f32, 2)

# 4. Lowe's Ratio Test를 통한 1차 필터링
good_matches = []
for i in range(len(des1)):
    if distances[i][0] < 0.5 * distances[i][1]:
        m = cv2.DMatch(_queryIdx=i, _trainIdx=int(indices[i][0]),
                        _distance=distances[i][0])
        good_matches.append(m)

print(f"Faiss 매칭 완료 | Ratio Test 통과: {len(good_matches)}개")
```

RANSAC 실습

5. RANSAC + Homography 계산 및 최종 시각화

if len(good_matches) > 4:

```
src_pts = np.float32([kp1[m.queryIdx].pt for m in good_matches]).reshape(-1, 1, 2)
```

```
dst_pts = np.float32([kp2[m.trainIdx].pt for m in good_matches]).reshape(-1, 1, 2)
```

RANSAC으로 homography 행렬(M) 계산

```
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 3.0)
```

```
matches_mask = mask.ravel().tolist()
```

Inliers만 추출

```
inlier_matches = [good_matches[i] for i in range(len(matches_mask)) if  
matches_mask[i] == 1]
```

```
print(f"RANSAC 검증 완료 | 최종 인라이어(Inliers): {len(inlier_matches)}개")
```

```
res_img = cv2.drawMatches(img1_gray, kp1, img2_gray, kp2, inlier_matches[:500],  
None,
```

```
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

```
print("\n[RANSAC 매칭 결과]")
```

```
cv2_imshow(res_img)
```

else:

```
print("매칭점이 부족하여 호모그래피를 계산할 수 없습니다.")
```

```
}
```

□ □

내용 정리

- 특징 검출 및 기술 알고리즘(SIFT, ORB)
- 매칭 방법 및 관련 library
- 매칭의 성능 측정 지표

다음 주 강의 내용

- 스테레오 영상의 기하 구조
- 스테레오 영상으로부터 깊이 추정 방법